

워크플로우 코드 왜 항상 복잡할까?

Temporal이 보여준 다른 방법

정승주

자기소개

- 정승주
- 백엔드 엔지니어
- 두번째 스프링캠프 발표!
- 잘 부탁드립니다♡♡



목차

01

워크플로우는 왜 항상 복잡할까

고객 보상 시나리오 - 비즈니스 코드가 파묻히는 과정

02

Temporal이 제안하는 방식

실패가 없는 것처럼 짜고 나머지는 Temporal이 맡는다

03

어떻게 가능한가? (Durable Execution의 원리)

Event History · Replay · Proxy — 멈췄다 그 자리에서 이어지는 원리

04

그래서 무엇이 남았나

비즈니스 시맨틱만 남은 코드와 복잡도의 행방

오늘의 주인공

Temporal

수백 줄이 되는 워크플로우 코드,
비즈니스 코드만 남긴다

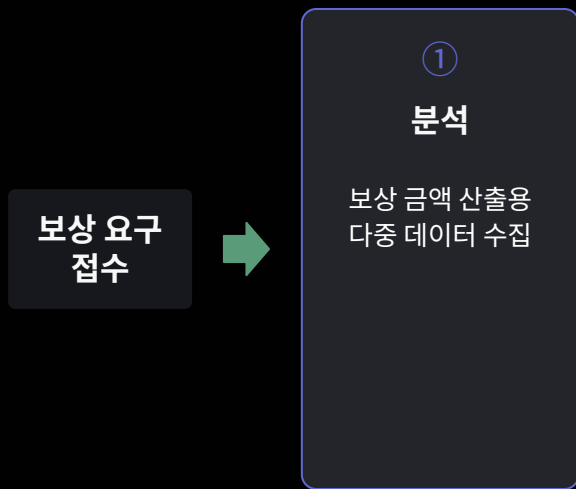
워크플로우는 왜 항상 복잡할까?

고객 보상 워크플로우, 직접 만들어 봅시다

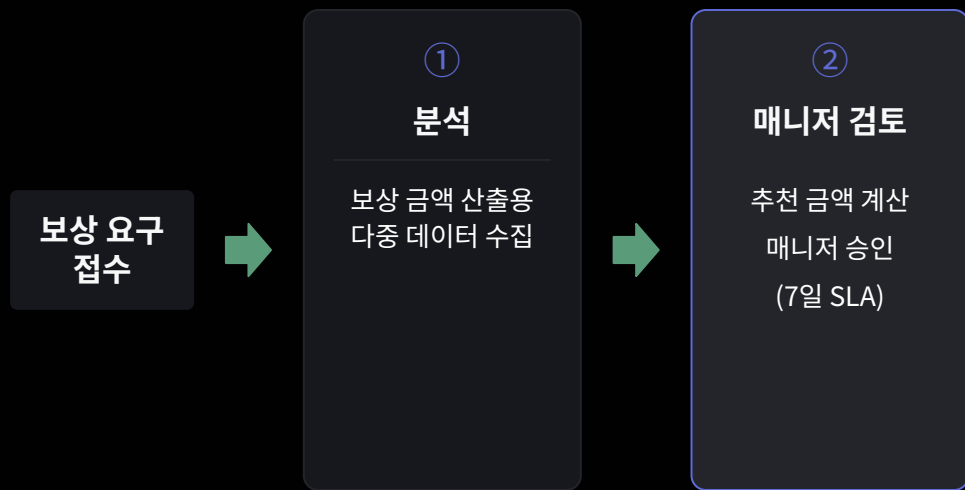
고객 보상 플로우

보상 요구
접수

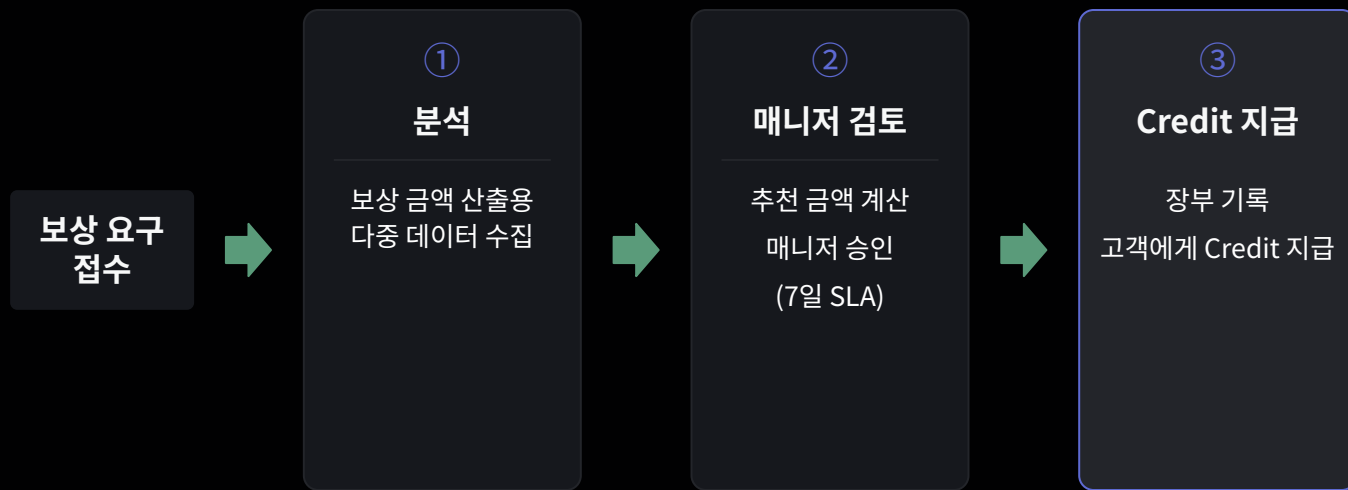
고객 보상 플로우



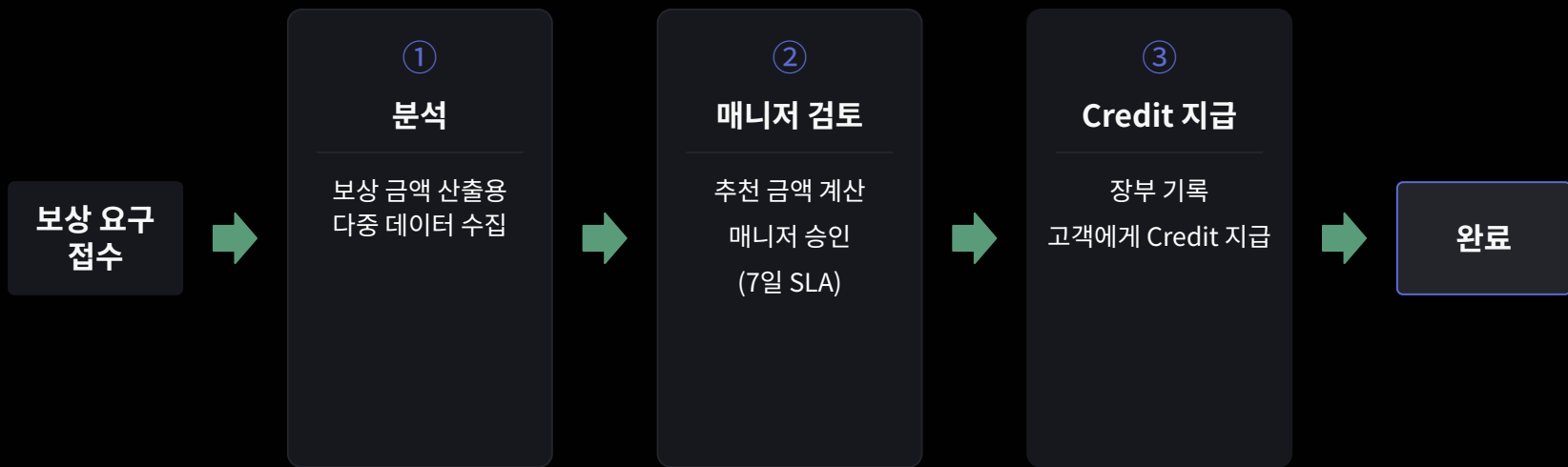
고객 보상 플로우



고객 보상 플로우



고객 보상 플로우

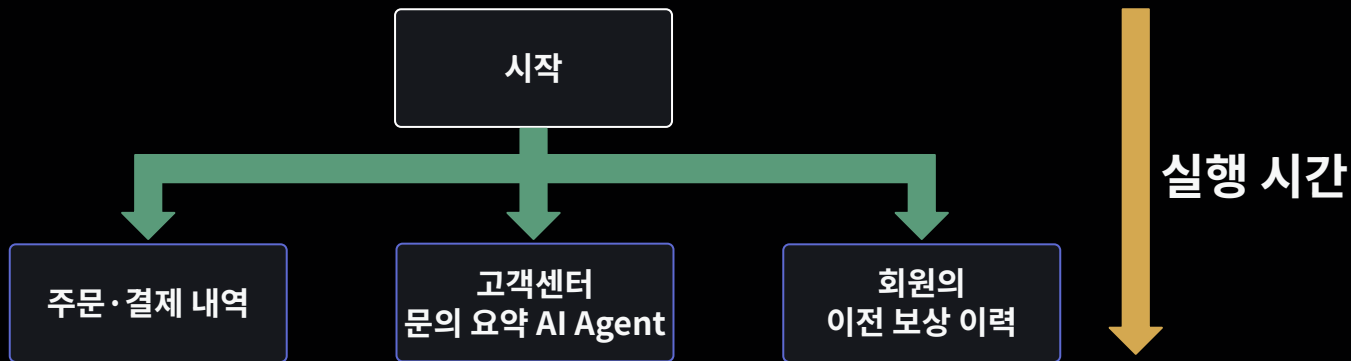


첫번째 단계

데이터 분석

분석

FAN OUT으로 병렬 수행



단순한 병렬은 쉽다

병렬의 각 갈래가 다르게 처리되어야 한다면?

1. 재시도

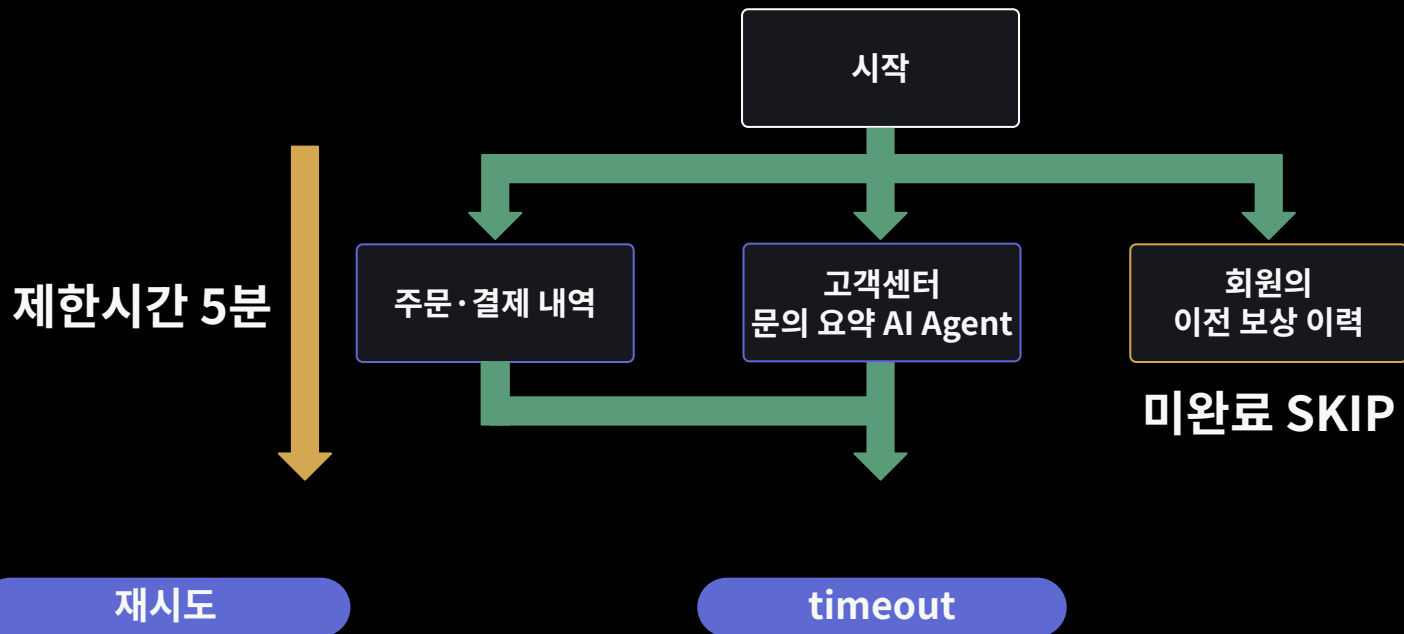
서비스마다 다른 정책



재시도

2. 제한시간

제한 시간



3. 결과의 영속화

실패는 언제나 발생한다



재시도

timeout

결과의 영속화

3. 결과의 영속화

실패는 언제나 발생한다



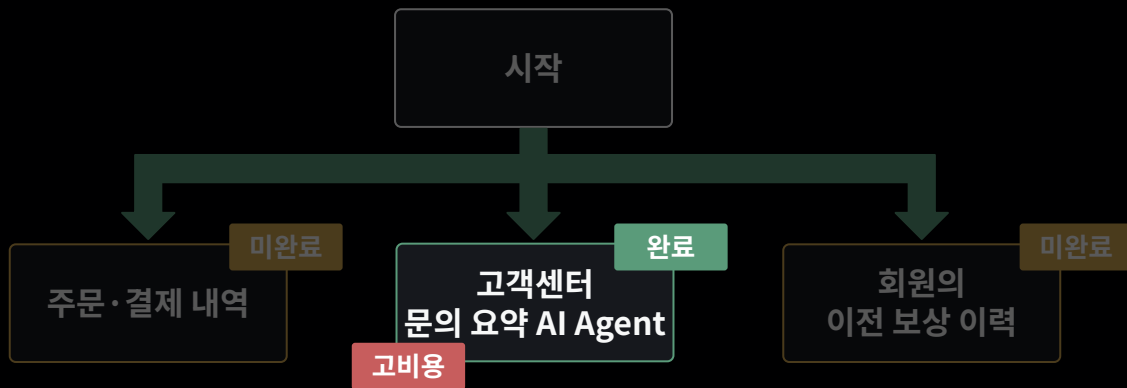
재시도

timeout

결과의 영속화

3. 결과의 영속화

실패는 언제나 발생한다



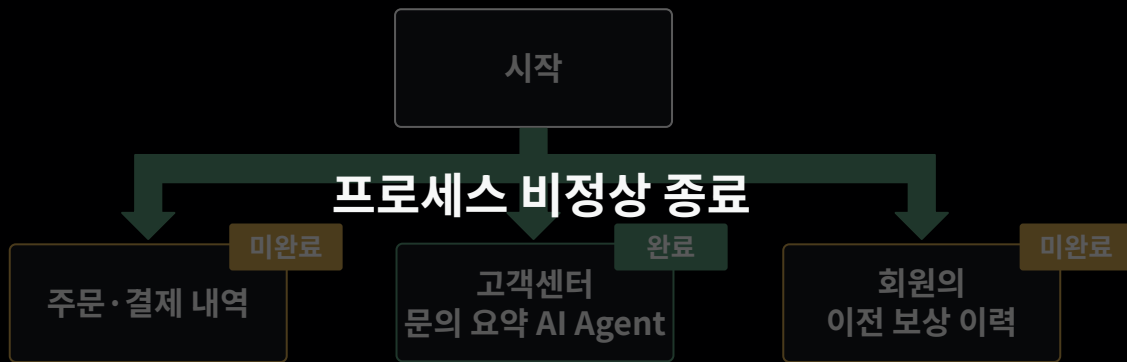
재시도

timeout

결과의 영속화

3. 결과의 영속화

실패는 언제나 발생한다



재시도

timeout

결과의 영속화

3. 결과의 영속화

실패는 언제나 발생한다



재시도

timeout

결과의 영속화

3. 결과의 영속화

실패는 언제나 발생한다



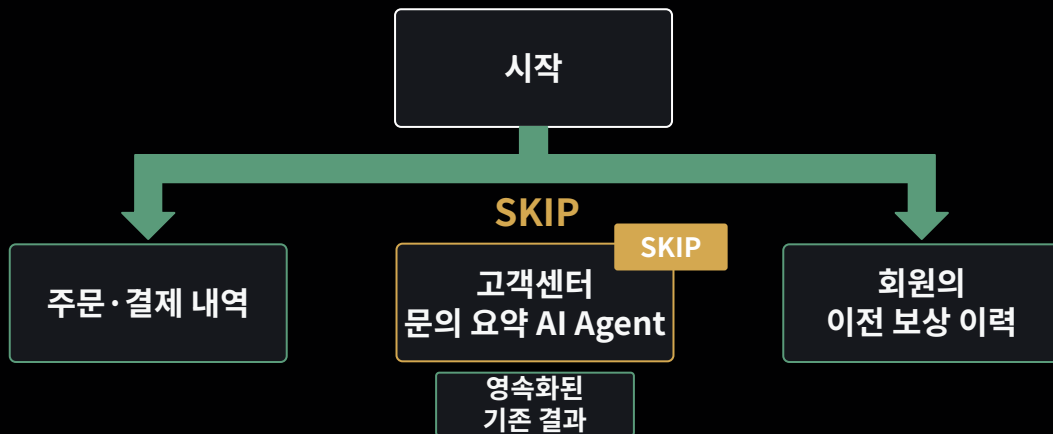
재시도

timeout

결과의 영속화

3. 결과의 영속화

기존 결과를 재사용 한다



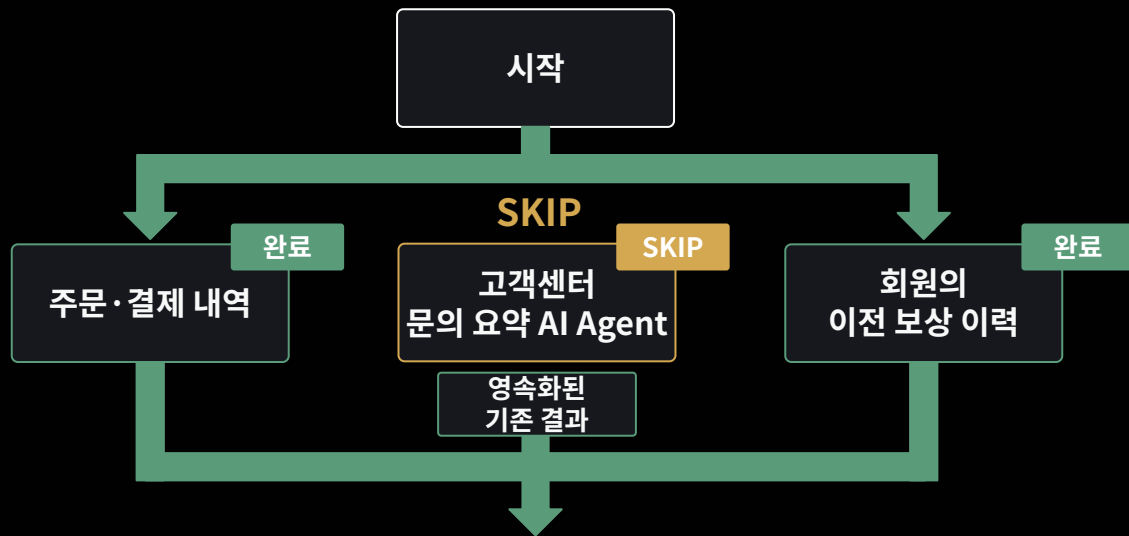
재시도

timeout

결과의 영속화

3. 결과의 영속화

기존 결과를 재사용 한다



재시도

timeout

결과의 영속화

지금까지 드러난 기능들

비즈니스 로직 외 무엇이 필요했나?

분석 - 3가지 추가 기능

비즈니스 로직 외 3가지 기능 필요

재시도

결과의 영속화

timeout

두번째 단계

매니저 승인

승인

보상 금액 평가 및 승인



승인 - 매니저 승인

긴 승인 대기 기간



최대 7일 승인 대기

승인 - 매니저 승인

긴 승인 대기 기간



최대 7일 승인 대기
미승인 시 실패 처리

승인 - 매니저 승인

긴 승인 대기 기간

함수에서 대기?



최대 7일 승인 대기

승인 - 매니저 승인

긴 승인 대기 기간

함수에서 대기?
불가능



최대 7일 승인 대기

승인 - 매니저 승인

긴 승인 대기 기간

함수에서 대기?
불가능



기간 만료?

최대 7일 승인 대기

승인 - 매니저 승인

기간 대기 기능

함수에서 대기?
불가능



최대 7일 승인 대기

기간 만료?
타이머 및 만료 처리 기능 필요

타이머

승인 - 매니저 승인

승인 신호 처리 기능

함수에서 대기?
불가능



최대 7일 승인 대기

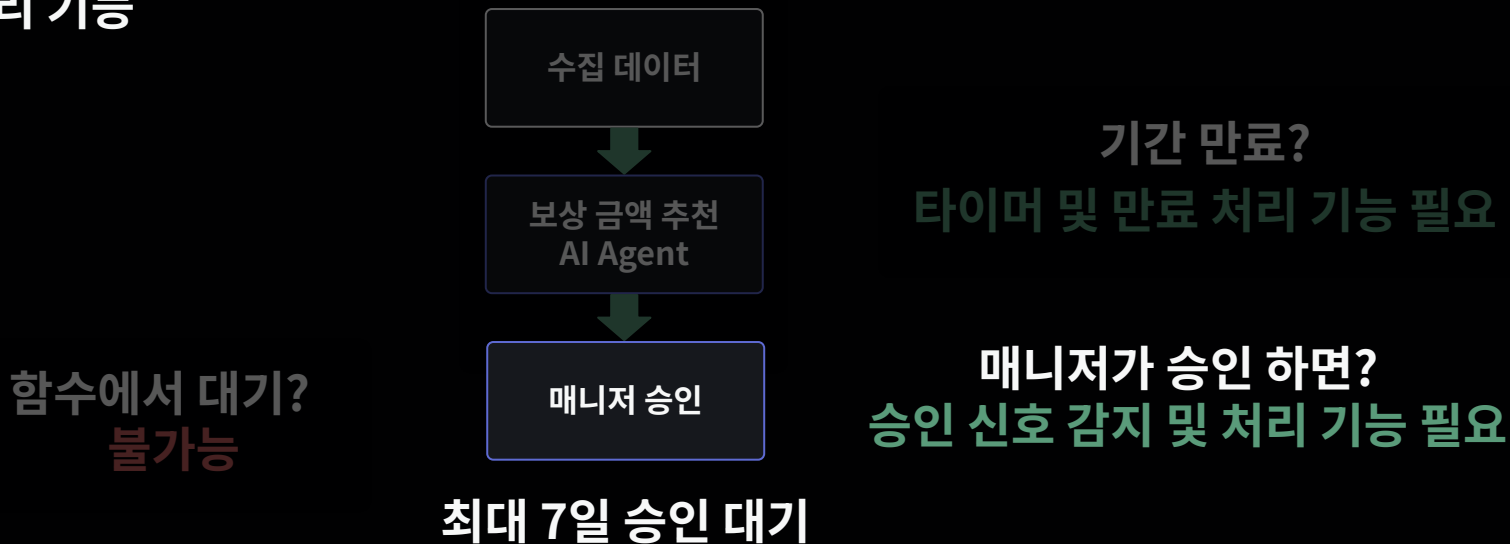
기간 만료?
타이머 및 만료 처리 기능 필요

매니저가 승인 하면?

타이머

승인 - 매니저 승인

승인 신호 처리 기능



타이머

신호 처리

승인 - 2가지 추가 기능

비즈니스 로직 외 2가지 기능 필요

타이머

신호 처리

추가 기능

비즈니스 로직 외 5가지 기능 필요

타이머

신호 처리

결과의 영속화

timeout

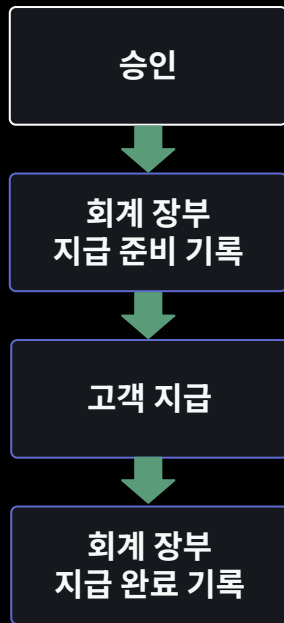
재시도

마지막 단계

크레딧을 지급합니다.

지급 - 고객 지급

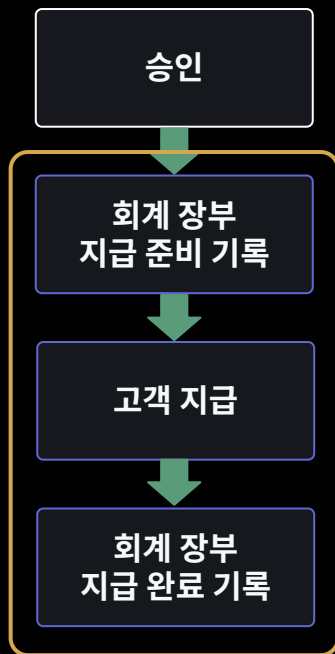
회계 장부 기록 및 고객에게 지급



지급 - 고객 지급

All or Nothing

부수 효과를 일으키는 연산들

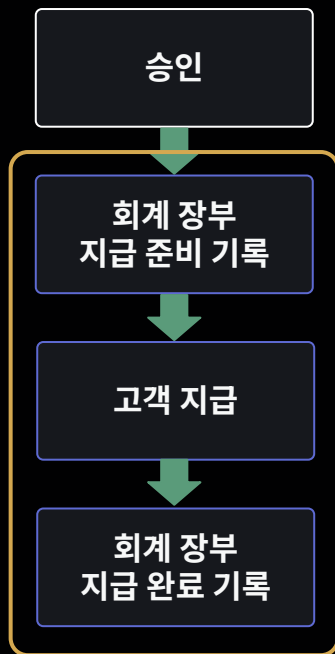


지급 - 고객 지급

All or Nothing

부수 효과를 일으키는 연산들

All or Nothing

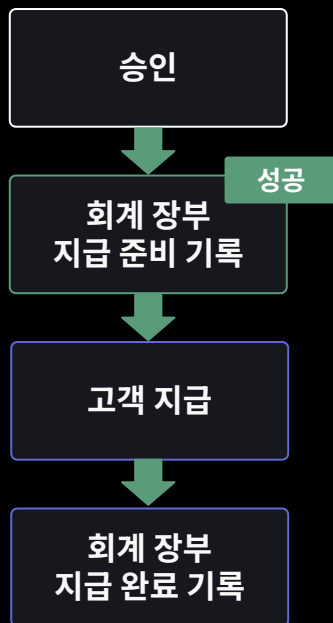


모두 수행 되거나

모두 수행 되지 않거나

지급 - 고객 지급

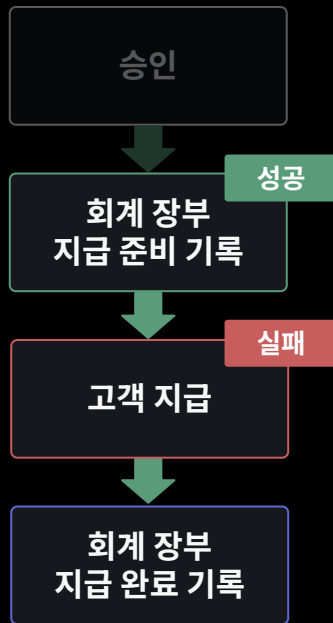
실패는 언제나 발생한다



All or Nothing

지급 - 고객 지급

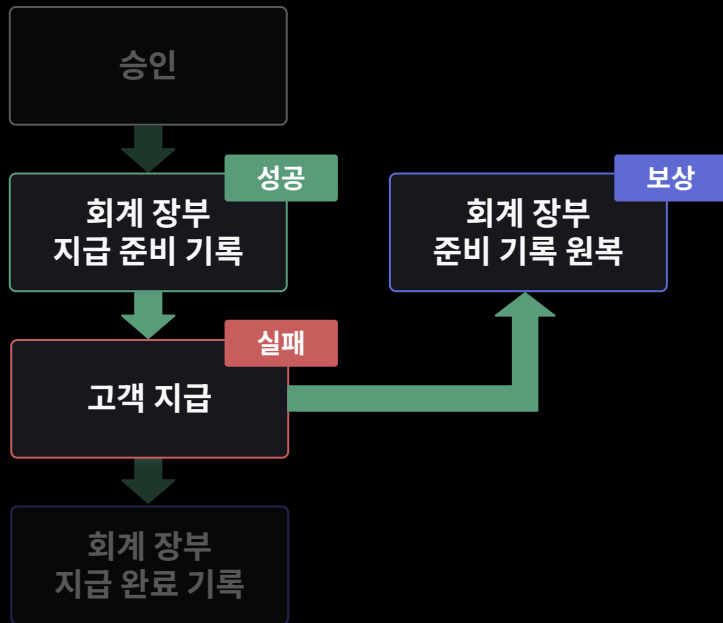
실패는 언제나 발생한다



All or Nothing

지급 - 고객 지급

보상을 통해 부수효과를 원복한다



All or Nothing

보상

지급 - 2가지 추가 기능

비즈니스 로직 외 2가지 기능 필요

All or Nothing

보상

추가 기능

비즈니스 로직 외 7가지 기능 필요

All or Nothing

타이머

timeout

재시도

보상

신호 처리

결과의 영속화

실제 필요한 요소들

```
class Saga {
```

```
...
```

```
}
```

```
class Task {
```

```
...
```

```
}
```

```
class DistributionLock {
```

```
...
```

```
}
```

```
class TaskContext {
```

```
...
```

```
}
```

```
class TaskFactory {
```

```
...
```

```
}
```

```
class TaskScheduler {
```

```
...
```

```
}
```

```
class SignalAcceptor {
```

```
...
```

```
}
```

```
class TaskWatcher {
```

```
...
```

```
}
```

```
void process(Ticket ticket) {  
    // 수집  
    var context = collectDataConcurrently(ticket)  
    // 평가  
    var amount = evaluate(context)  
    // 7일 대기 혹은 매니저 승인  
    await(Duration.ofDays(7), () -> approved)  
    if (approved) {  
        // 지급  
        recordAndPay(amount, ticket)  
    }  
}
```

```
class AsyncExecutor {
```

```
...
```

```
}
```

```
class Compensation {
```

```
...
```

```
}
```

```
class TaskExecutor {
```

```
...
```

```
}
```

```
enum Status {
```

```
...
```

```
}
```

```
class TaskContextRepository {
```

```
...
```

단순한 워크플로우들은?

더 복잡한 시나리오라면?

더 많고 정교한 기능들 필요

우리가 하고 싶었던 것은?

우리가 하고 싶었던 것은?

고객 데이터를 수집해서 금액평가 후 매니저 승인 받아 지급한다.

목차

01

워크플로우는 왜 항상 복잡할까

고객 보상 시나리오 - 비즈니스 코드가 파묻히는 과정

02

Temporal이 제안하는 방식

실패가 없는 것처럼 짜고 나머지는 Temporal이 맡는다

03

어떻게 가능한가? (Durable Execution의 원리)

Event History · Replay · Proxy — 멈췄다 그 자리에서 이어지는 원리

04

그래서 무엇이 남았나

비즈니스 시맨틱만 남은 코드와 복잡도의 행방

**“Write code as if
failure doesn’t exist.”**

실패가 존재하지 않는 것처럼 코드를 짠다

**“You code your business logic.
We handle the rest.”**

당신은 비즈니스 로직을 짠다 - 나머지는 우리가 한다

추가 기능

비즈니스 로직 외 7가지 기능 필요

All or Nothing

타이머

제한시간

재시도

보상

신호 처리

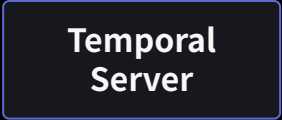
결과의 영속화

Temporal - 구성

4가지 요소

Temporal - 구성

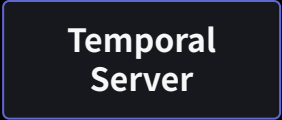
4가지 요소

A dark blue rounded rectangle with a thin light blue border, containing the text "Temporal Server" in white.

Temporal
Server

Temporal - 구성

4가지 요소



Temporal
Server



Worker

Temporal - 구성

4가지 요소

Temporal
Server

Worker

Workflow

Temporal - 구성

4가지 요소

Temporal
Server

Worker

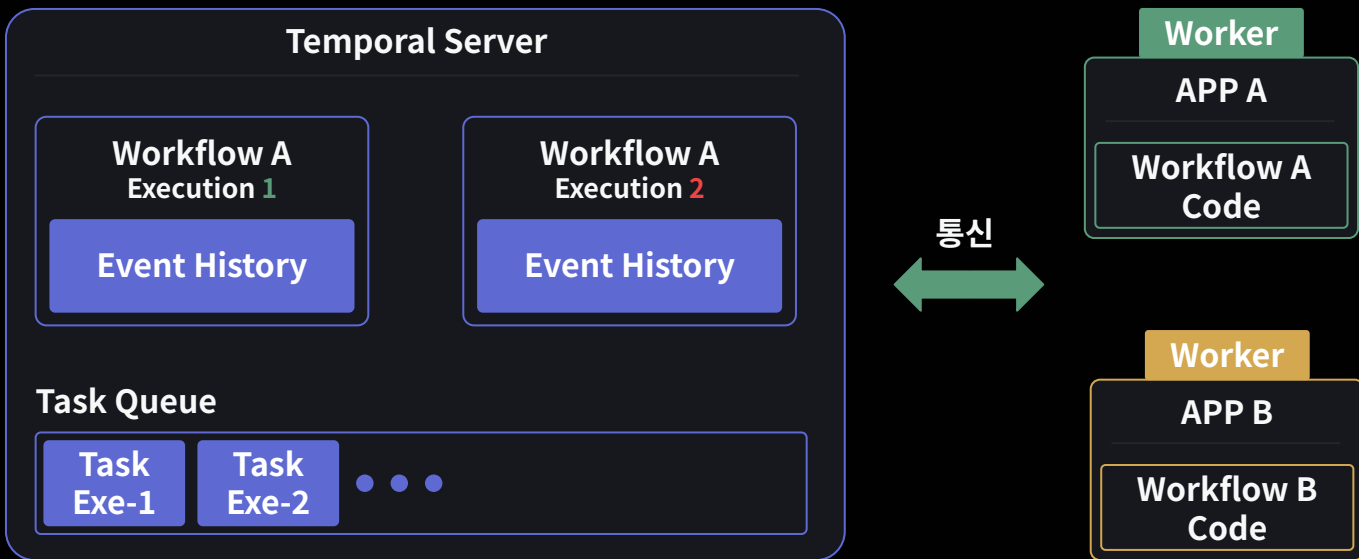
Workflow

Activity

Temporal Server + Worker

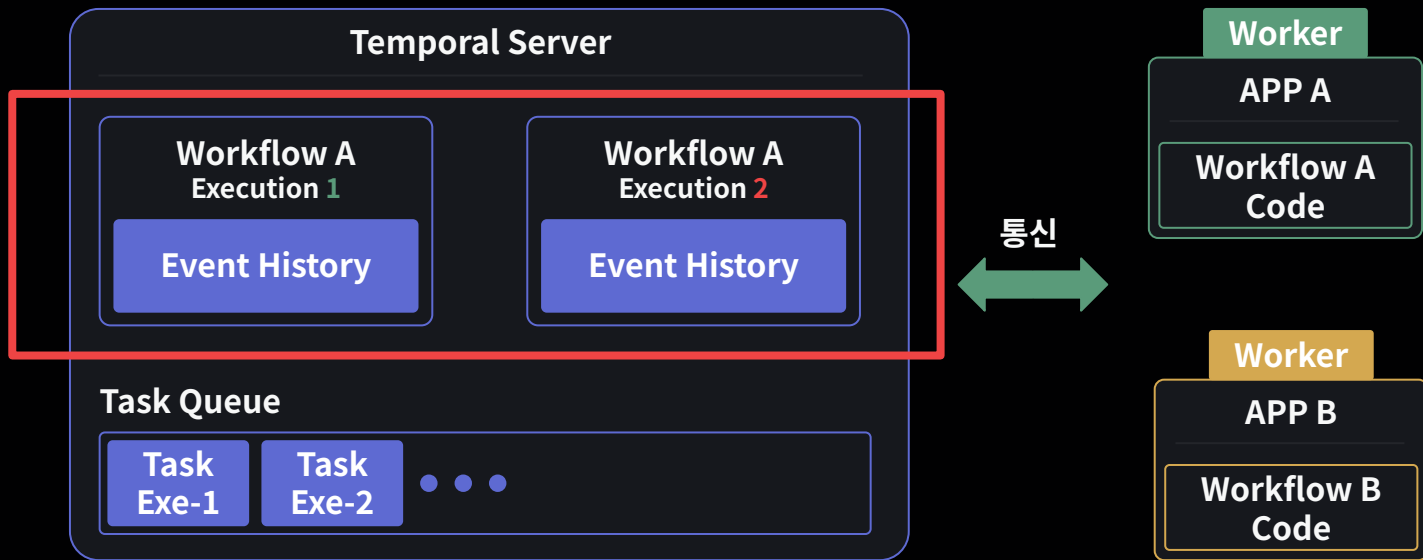
Temporal Server

중앙 조정자, 상태 저장소



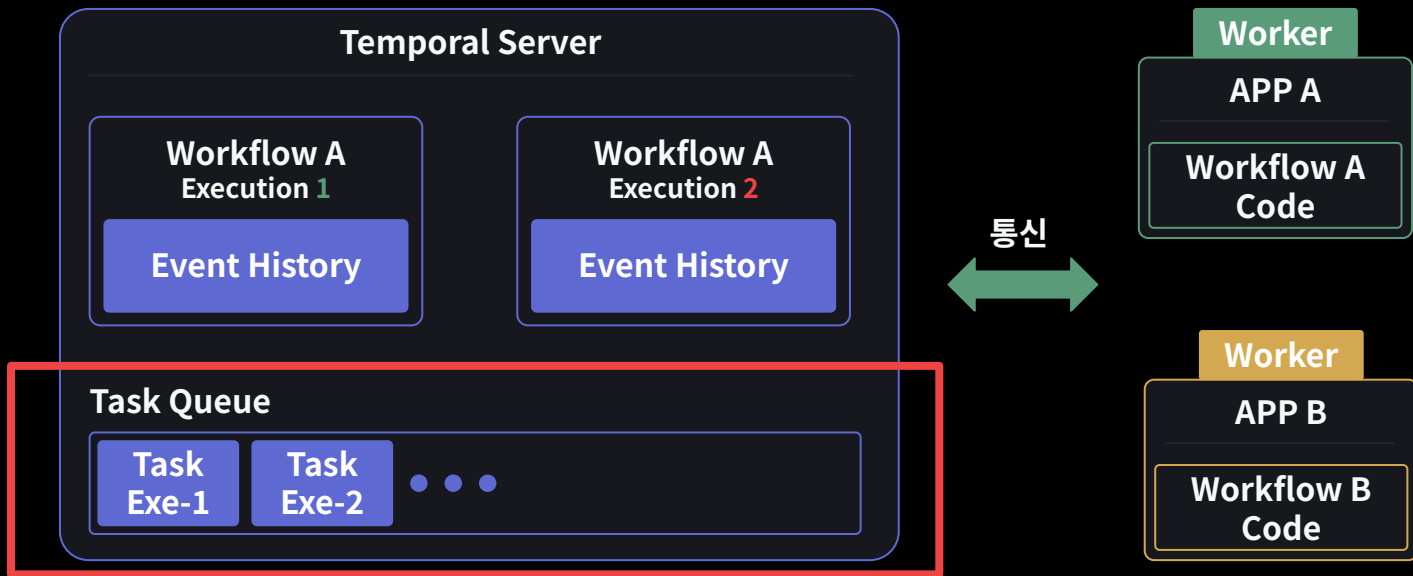
Temporal Server

중앙 조정자, 상태 저장소



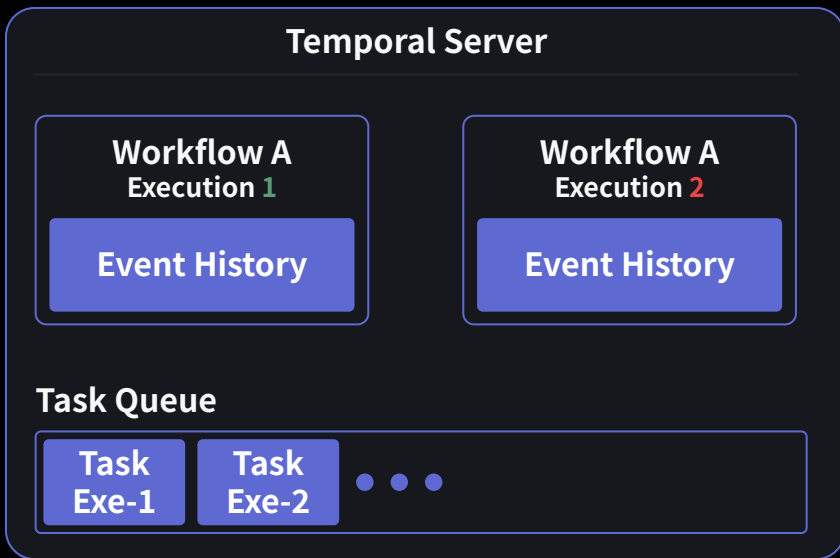
Temporal Server

중앙 조정자, 상태 저장소

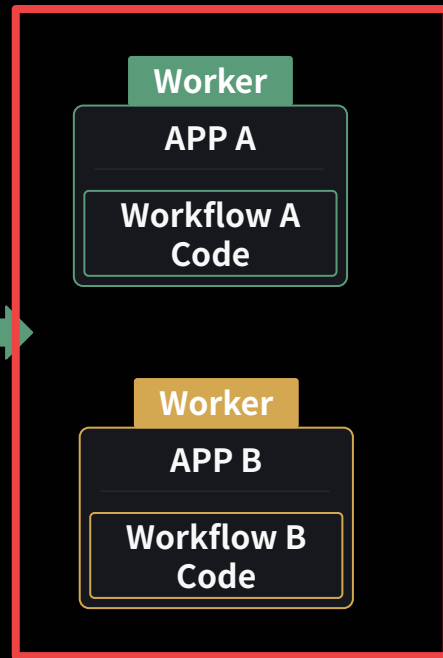


Temporal Server

중앙 조정자, 상태 저장소

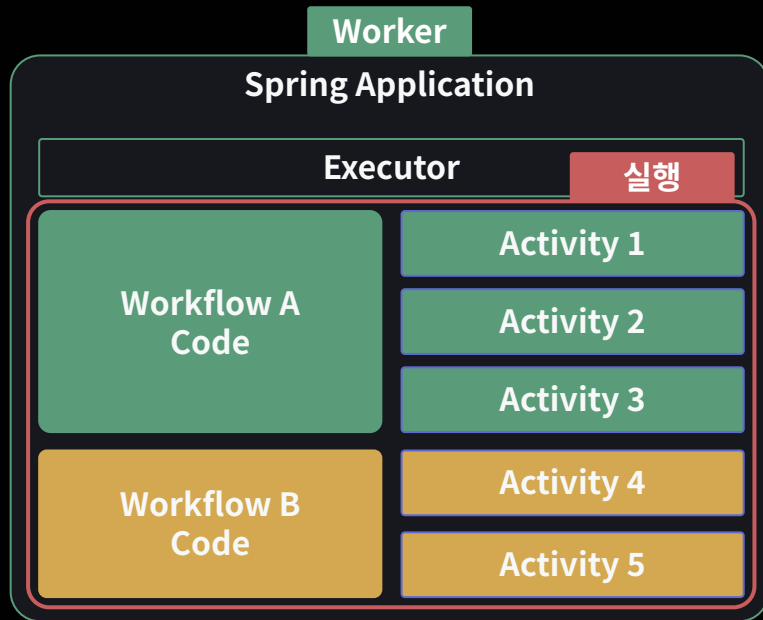
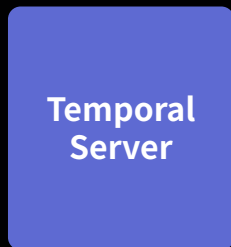


통신
↔



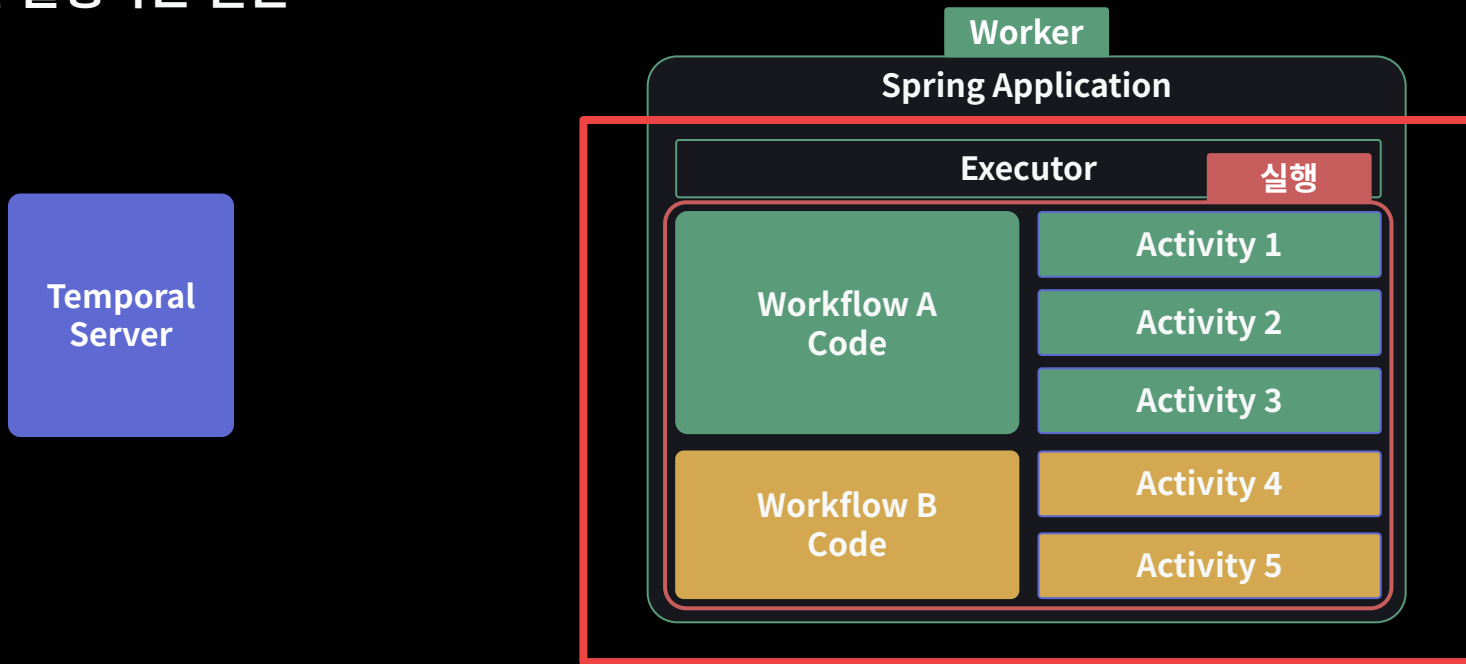
Worker

내 코드를 실행하는 일꾼



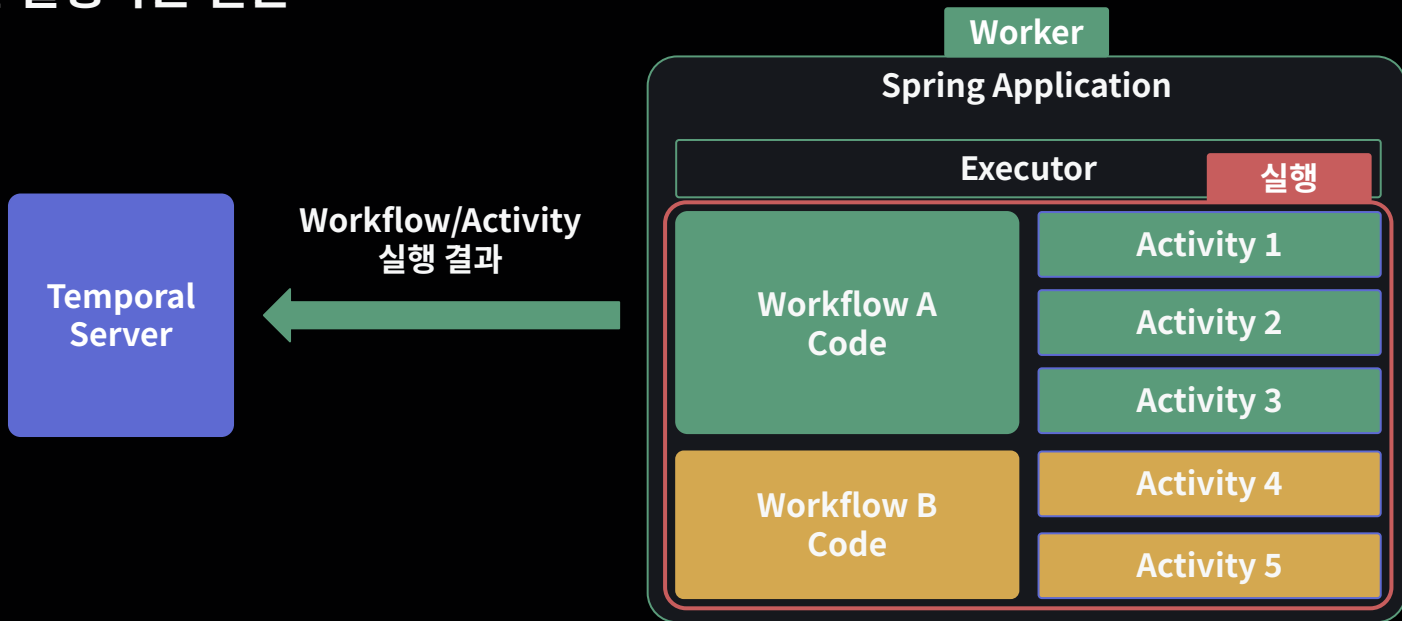
Worker

내 코드를 실행하는 일꾼



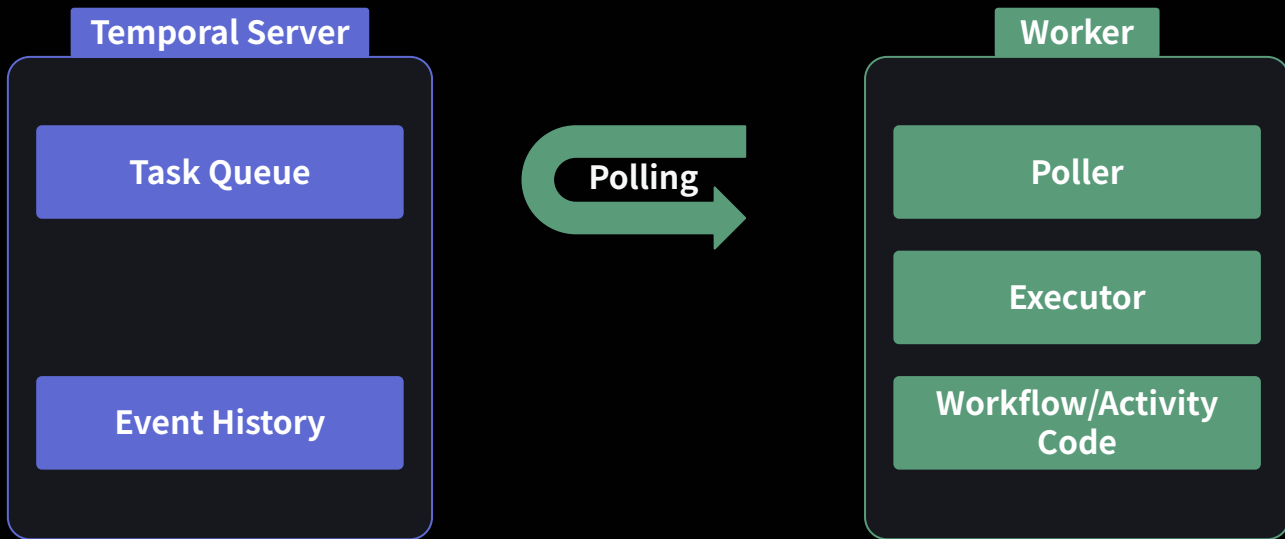
Worker

내 코드를 실행하는 일꾼



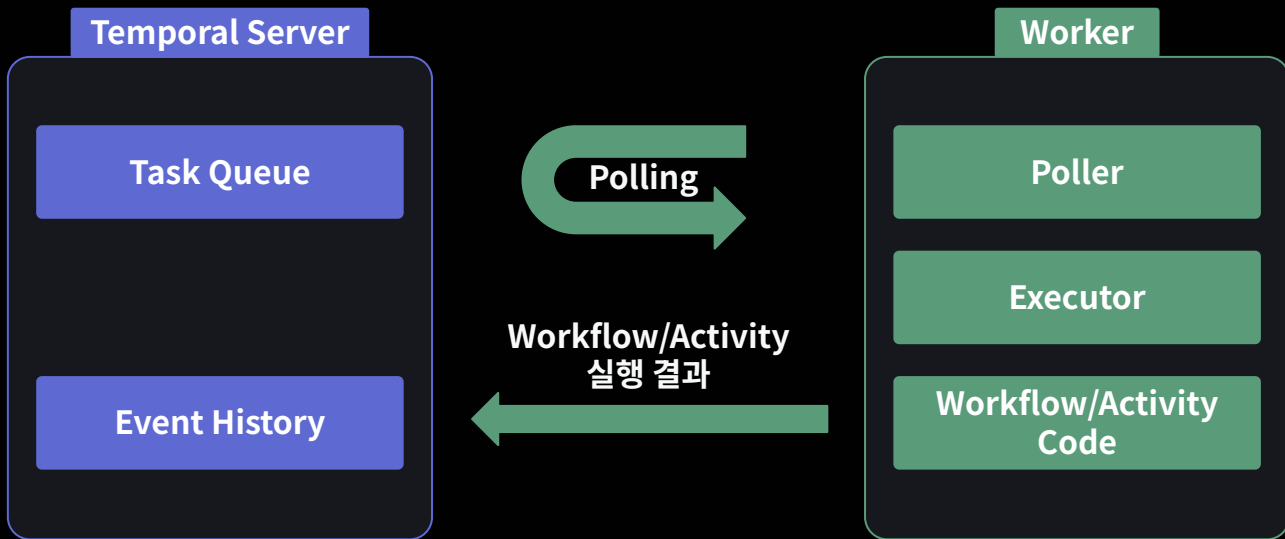
Temporal Server + Worker

어떻게 협업하나



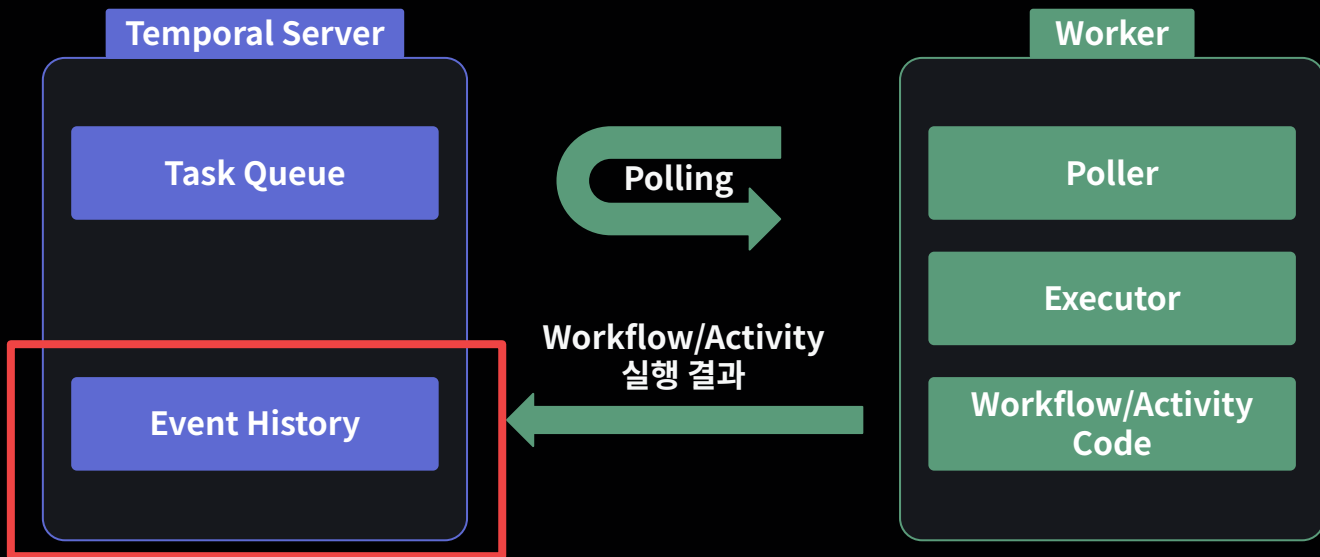
Temporal Server + Worker

어떻게 협업하나



Temporal Server + Worker

어떻게 협업하나



Workflow + Activity

Temporal - Workflow

| 일의 흐름만 명세

Temporal - Workflow

일의 흐름만 명세

데이터 수집

Temporal - Workflow

일의 흐름만 명세

데이터 수집



금액 평가

Temporal - Workflow

일의 흐름만 명세



Temporal - Workflow

일의 흐름만 명세



Temporal - Workflow

일의 흐름만 명세



무엇을 / 언제 / 어떤 순서로 진행할지를 명세

Temporal - Workflow

일의 흐름만 명세



일의 수행은 Activity에게 위임

Temporal - Workflow

실제 동작하는 코드로 명세

```
@WorkflowInterface
interface MyWorkflowInterface {

    @WorkflowMethod
    void process(MyCommand command)
}
```

Interface 선언

Temporal - Workflow

실제 동작하는 코드로 명세

```
@WorkflowInterface  
interface MyWorkflowInterface {  
  
    @WorkflowMethod  
    void process(MyCommand command)  
}
```

Interface 선언

Temporal - Workflow

실제 동작하는 코드로 명세

```
@WorkflowInterface
interface MyWorkflowInterface {

    @WorkflowMethod
    void process(MyCommand command)
}
```



```
class MyWorkflow implements MyWorkflowInterface {
    var actA = Workflow.newActivityStub(ActivityA.class)
    var actB = Workflow.newActivityStub(ActivityB.class)

    void process(MyCommand command) {
        // foo 수행
        var a = actA.foo(command.aArg)
        // 1일 대기
        Workflow.sleep(Duration.ofDays(1))
        // bar 수행
        actB.bar(a)
    }
}
```

Interface 선언

Interface 구현 및 흐름 선언

Temporal - Workflow

실제 동작하는 코드로 명세



동작하는 코드가 Workflow

=

```
class MyWorkflow implements MyWorkflowInterface {
    var actA = Workflow.newActivityStub(ActivityA.class)
    var actB = Workflow.newActivityStub(ActivityB.class)

    void process(MyCommand command) {
        // foo 수행
        var a = actA.foo(command.aArg)
        // 1일 대기
        Workflow.sleep(Duration.ofDays(1))
        // bar 수행
        actB.bar(a)
    }
}
```

Interface 구현 및 흐름 선언

Temporal - Workflow

실제 동작하는 코드로 명세



동작하는 코드가 Workflow

=

```
class MyWorkflow implements MyWorkflowInterface {  
    var actA = Workflow.newActivityStub(ActivityA.class)  
    var actB = Workflow.newActivityStub(ActivityB.class)  
  
    void process(MyCommand command) {  
        // foo 수행  
        var a = actA.foo(command.aArg)  
        // 1일 대기  
        Workflow.sleep(Duration.ofDays(1))  
        // bar 수행  
        actB.bar(a)  
    }  
}
```

Interface 구현 및 흐름 선언

Temporal - Workflow

실제 동작하는 코드로 명세



동작하는 코드가 Workflow

=

```
class MyWorkflow implements MyWorkflowInterface {  
    var actA = Workflow.newActivityStub(ActivityA.class)  
    var actB = Workflow.newActivityStub(ActivityB.class)  
  
    void process(MyCommand command) {  
        // foo 수행  
        var a = actA.foo(command.aArg)  
        // 1일 대기  
        Workflow.sleep(Duration.ofDays(1))  
        // bar 수행  
        actB.bar(a)  
    }  
}
```

Interface 구현 및 흐름 선언

Temporal - Workflow

실제 동작하는 코드로 명세



동작하는 코드가 Workflow

=

```
class MyWorkflow implements MyWorkflowInterface {  
    var actA = Workflow.newActivityStub(ActivityA.class)  
    var actB = Workflow.newActivityStub(ActivityB.class)  
  
    void process(MyCommand command) {  
        // foo 수행  
        var a = actA.foo(command.aArg)  
        // 1일 대기  
        Workflow.sleep(Duration.ofDays(1))  
        // bar 수행  
        actB.bar(a)  
    }  
}
```

Interface 구현 및 흐름 선언

Temporal - Workflow

Workflow 객체 생성 후 사용

```
// private WorkflowClient workflowClient

var workflow = workflowClient.newWorkflowStub(
    MyWorkflow.class,
    WorkflowOptions.newBuilder()
        // 여러 Workflow Option ...
        .build()
)

// @WorkflowMethod가 등록된 함수
workflowClient.start(workflow::process, command)
```

객체 생성 후 WorkflowClient로
@WorkflowMethod가 등록된 함수를 전달하여 실행

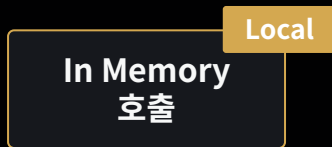
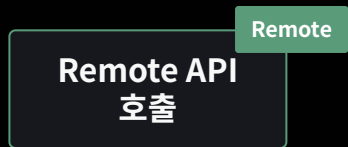
Temporal - Activity

실제 일을 수행

```
class MyWorkflow implements MyWorkflowInterface {  
    var actA = Workflow.newActivityStub(ActivityA.class)  
    var actB = Workflow.newActivityStub(ActivityB.class)  
  
    void process(MyCommand command) {  
        // foo 수행  
        var a = actA.foo(command.aArg)  
        // 1일 대기  
        Workflow.sleep(Duration.ofDays(1))  
        // bar 수행  
        actB.bar(a)  
    }  
}
```


Temporal - Activity

실제 일을 수행



Remote든, In Memory든

Temporal - Activity

실제 일을 수행



Remote든, In Memory든
부수 효과가 있든 없든

Temporal - Activity

실제 일을 수행



Remote든, In Memory든
부수 효과가 있든 없든
일로서 실제 수행하는 것이 Activity

Temporal - Activity

비결정적 로직의 구현 위치

```
@ActivityInterface
interface OrderActivityInterface {

    @ActivityMethod
    Order fetchOrder(int orderId)
}
```

Interface 선언

Temporal - Activity

비결정적 로직의 구현 위치

```
@ActivityInterface  
interface OrderActivityInterface {  
  
    @ActivityMethod  
    Order fetchOrder(int orderId)  
}
```

Interface 선언

Temporal - Activity

비결정적 로직의 구현 위치

```
@ActivityInterface
interface OrderActivityInterface {

    @ActivityMethod
    Order fetchOrder(int orderId)
}
```



```
class OrderActivity implements OrderActivityInterface {

    private RestClient client

    Order fetchOrder(int orderId) {
        return client.get()
            .uri("/orders/{id}", orderId)
            .retrieve()
            .body(Order.class)
    }
}
```

Interface 선언

Interface 구현 및
비결정적 논리 구현

Temporal - Activity

여러 옵션과 함께 Activity 객체 생성

```
OrderActivity activity = Workflow.newActivityStub(  
    OrderActivity.class,  
    ActivityOptions.newBuilder()  
        .setRetryOption(  
            RetryOptions.newBuilder()  
                .setInitialInterval(Duration.ofSeconds(1))  
                .setBackoffCoefficient(2.0)  
                .build()  
        )  
        .build()  
);  
// @ActivityMethod가 붙은 함수  
activity.fetchOrder(orderId)
```

Workflow로 객체 생성 후
@ActivityMethod가 등록된 함수를 호출하여 실행

Temporal - Spring

Workflow for Spring

Spring Properties

```
spring:  
  application:  
    name: my-worker  
  temporal:  
    connection:  
      target: 127.0.0.1:7233  
    namespace: default  
    workers-auto-discovery:  
      packages:  
        - com.example.my.workflows  
        - com.example.my.activities  
    workers:  
      - task-queue: my-task-queue
```

Workflow Class

```
@WorkflowImpl  
class MyWorkflow implements MyWorkflowInterface {  
  
    ...  
  
}
```

Activity Class

```
@ActivityImpl  
class MyActivity implements MyActivityInterface {  
  
    ...  
  
}
```


Temporal - Spring

Activity for Spring

Activity Class

```
@Component
@ActivityImpl
class MyActivity implements MyActivityInterface {

    // 생성자 주입
    private final ServiceA serviceA;
    private final ServiceB serviceB;

    ...

}
```

Temporal - Spring

Activity for Spring

Activity Class

```
@Component
@ActivityImpl
class MyActivity implements MyActivityInterface {

    // 생성자 주입
    private final ServiceA serviceA;
    private final ServiceB serviceB;

    ...

}
```

얇은 껍질

Service Class

```
class DefaultServiceA implements ServiceA {

    ...

}

class DefaultServiceB implements ServiceB {

    ...

}
```

온전한 본질

왜 둘로 나뉘었을까?

단순한 역할/책임 분리?

단순한 역할/책임 분리?

좀 더 복잡한 목적!

Temporal - Workflow

재실행 되는 Workflow 코드

```
void process(MyCommand command) {  
    // foo 수행  
    var a = actA.foo(command.aArg)  
  
    // 1일 대기  
    Workflow.sleep(Duration.ofDays(1))  
  
    // bar 수행  
    actB.bar(a)  
}
```

Temporal - Workflow

재실행 되는 Workflow 코드

```
void process(MyCommand command) {  
    // foo 수행  
    var a = actA.foo(command.aArg)  
  
    // 1일 대기  
    Workflow.sleep(Duration.ofDays(1))  
  
    // bar 수행  
    actB.bar(a)  
}
```

Workflow 코드는 여러번 재실행 됨

Temporal - Workflow

재실행 되는 Workflow 코드

```
void process(MyCommand command) {  
    // foo 수행  
    var a = actA.foo(command.aArg)  
  
    // 1일 대기  
    Workflow.sleep(Duration.ofDays(1))  
  
    // bar 수행  
    actB.bar(a)  
}
```


Temporal - Workflow

재실행 되는 Workflow 코드

```
void process(MyCommand command) {  
    // foo 수행  
    var a = actA.foo(command.aArg)  
  
    // 1일 대기  
    Workflow.sleep(Duration.ofDays(1))  
  
    // bar 수행  
    actB.bar(a)  
}
```

**1일 대기 수행 시
함수가 종료됨**
(park 됐다가,
캐시, 스레드 한도 도달 시
DestroyWorkflowThreadError로
정리)

Temporal - Workflow

재실행 되는 Workflow 코드

1일 뒤에 처음부터
다시 시작



```
void process(MyCommand command) {  
    // foo 수행  
    var a = actA.foo(command.aArg)  
  
    // 1일 대기  
    Workflow.sleep(Duration.ofDays(1))  
  
    // bar 수행  
    actB.bar(a)  
}
```



1일 대기 수행 시
함수가 종료됨
(park 됐다가,
캐시, 스레드 한도 도달 시
DestroyWorkflowThreadError로
정리)

Temporal - Workflow

재실행 되는 Workflow 코드

처음부터
다시 시작



```
void process(MyCommand command) {  
    // foo 수행  
    var a = actA.foo(command.aArg)  
  
    // 1일 대기  
    Workflow.sleep(Duration.ofDays(1))  
  
    // bar 수행  
    actB.bar(a)  
}
```

여러 가지 사유로 처음부터 재실행 가능

Temporal - Workflow

재실행 되는 Workflow 코드

```
void process(MyCommand command) {  
    // foo 수행  
    var a = actA.foo(command.aArg)  
  
    // 1일 대기  
    Workflow.sleep(Duration.ofDays(1))  
  
    // bar 수행  
    actB.bar(a)  
}
```

만약 재실행 때마다 Activity 결과가 다르다면?

Temporal - Activity

결정적으로 호출 되어야 하는 Activity

처음 시작
(1회차)

1회차
a = 10

```
void process(MyCommand command) {  
    // foo 수행  
    var a = actA.foo(command.aArg)  
  
    // 1일 대기  
    Workflow.sleep(Duration.ofDays(1))  
  
    // bar 수행  
    actB.bar(a)  
}
```

Temporal - Activity

결정적으로 호출 되어야 하는 Activity

1일 뒤 재시작
(2회차)

1회차
a = 10

```
void process(MyCommand command) {  
    // foo 수행  
    var a = actA.foo(command.aArg)  
  
    // 1일 대기  
    Workflow.sleep(Duration.ofDays(1))  
  
    // bar 수행  
    actB.bar(a)  
}
```

2회차
a = 20

Temporal - Activity

결정적으로 호출 되어야 하는 Activity

```
void process(MyCommand command) {  
    // foo 수행  
    var a = actA.foo(command.aArg)  
  
    // 1일 대기  
    Workflow.sleep(Duration.ofDays(1))  
  
    // bar 수행  
    actB.bar(a)  
}
```

1회차
a = 10

actB는 어떤 값을 써야 하는가?

2회차
a = 20

Temporal - Activity

결정적으로 호출 되어야 하는 Activity

```
void process(MyCommand command) {  
    // foo 수행  
    var a = actA.foo(command.aArg)  
  
    // 1일 대기  
    Workflow.sleep(Duration.ofDays(1))  
  
    // bar 수행  
    actB.bar(a)  
}
```

1회차
a = 10

항상 1회차의 값을 재사용해야 함
즉, 결정적이어야 한다

2회차
a = 20

Temporal - 결정적

| 결정적이란?

Random

수행 시마다
다른 값 가능성

Temporal - 결정적

| 결정적이란?

Random

수행 시마다
다른 값 가능성

DB 조회

데이터 상태 마다
다른 값 조회

Temporal - 결정적

| 결정적이란?

Random

수행 시마다
다른 값 가능성

DB 조회

데이터 상태 마다
다른 값 조회

부수 효과

평가 마다
부수효과 발생

Temporal - 결정적

| 결정적이란?

매번 다른 값 평가 가능
비결정적

Random

수행 시마다
다른 값 가능성

DB 조회

데이터 상태마다
다른 값 조회

부수 효과

평가마다
부수효과 발생

Workflow / Activity

모순

Workflow는 결정적



```
void process(MyCommand command) {  
    // foo 수행  
    var a = actA.foo(command.aArg)  
  
    // 1일 대기  
    Workflow.sleep(Duration.ofDays(1))  
  
    // bar 수행  
    actB.bar(a)  
}
```

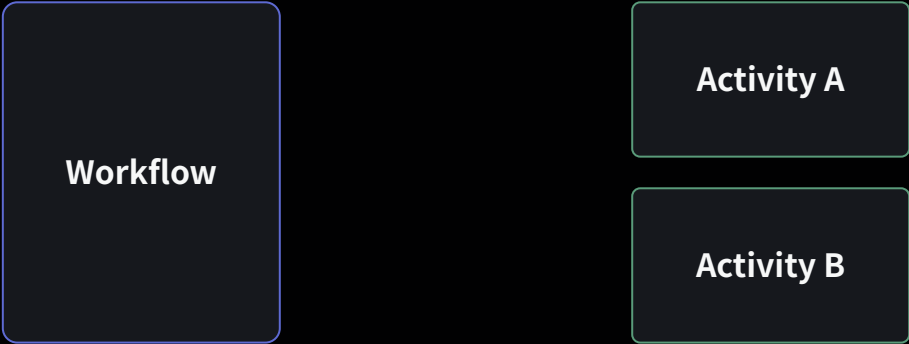


Activity는 비결정적

**Activity는 비결정적
Workflow가 결정적으로 호출**

Temporal - Workflow

Workflow가 결정적으로 호출



Workflow

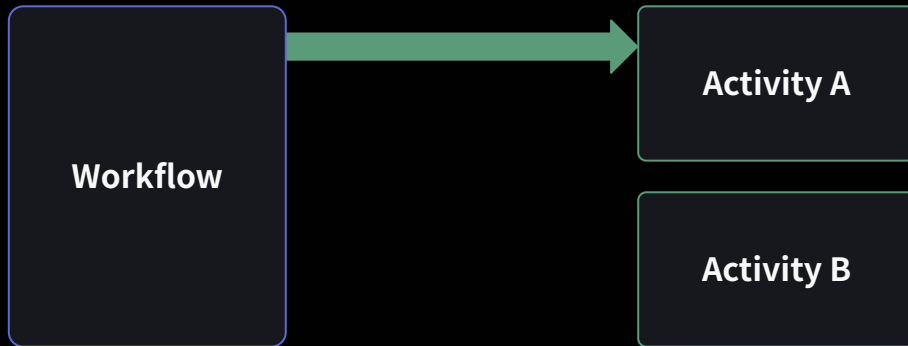
Activity A

Activity B

Temporal - Workflow

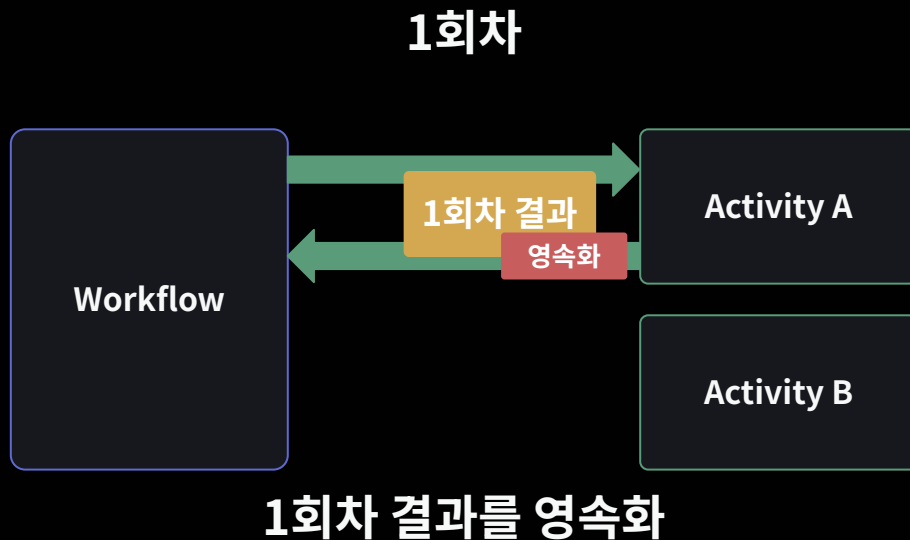
Workflow가 결정적으로 호출

1회차



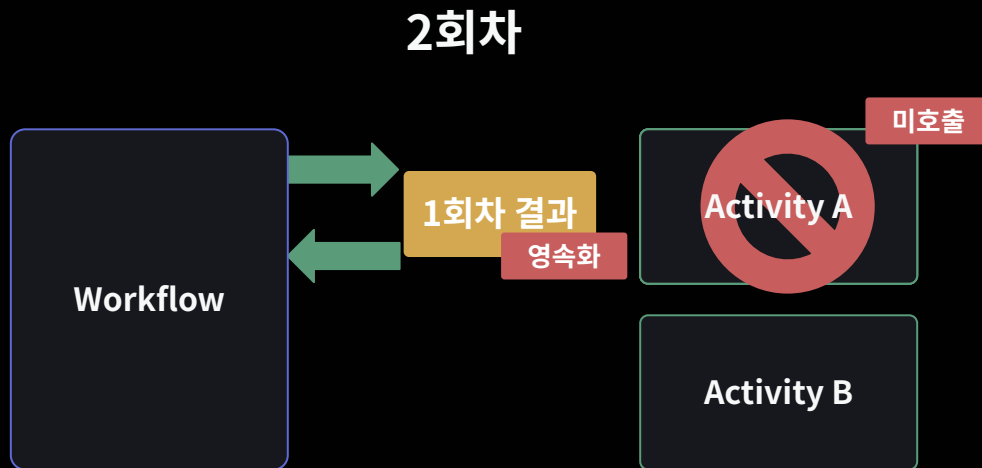
Temporal - Workflow

Workflow가 결정적으로 호출



Temporal - Workflow

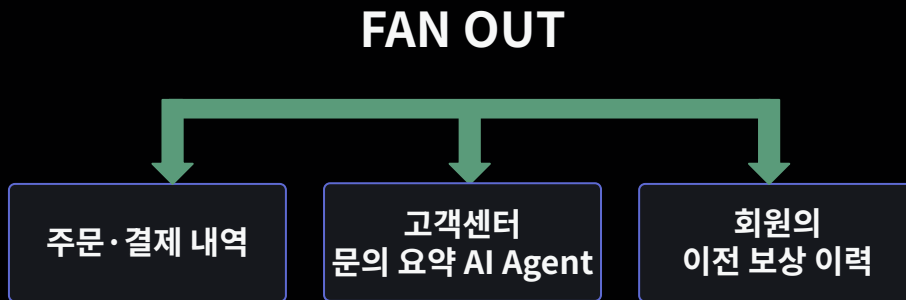
Workflow가 결정적으로 호출



고객 보상 워크플로우 Temporal Code로 보기

Temporal - Workflow

데이터 조회 부분 코드



Temporal - Code

Activity별 Retry 전략 설정

```
var orderPolicy = RetryOptions.newBuilder()  
    .setInitialInterval(Duration.ofSeconds(2))  
    .setBackoffCoefficient(1.5)  
    .setMaximumAttempts(5)  
    .build()
```

```
var agentPolicy = RetryOptions.newBuilder()  
    .setInitialInterval(Duration.ofSeconds(10))  
    .setBackoffCoefficient(1.2)  
    .setMaximumInterval(Duration.ofSeconds(10))  
    .build()
```

```
var prevPolicy = RetryOptions.newBuilder()  
    .setInitialInterval(Duration.ofSeconds(1))  
    .setMaxAttempts(2)  
    .build()
```

주문 API 호출
재시도 전략

Agent 호출
재시도 전략

과거 보상 API
호출 재시도 전략

Temporal - Code

Activity별 Retry 전략 설정

```
var orderPolicy = RetryOptions.newBuilder()  
    .setInitialInterval(Duration.ofSeconds(2))  
    .setBackoffCoefficient(1.5)  
    .setMaximumAttempts(5)  
    .build()
```

```
var agentPolicy = RetryOptions.newBuilder()  
    .setInitialInterval(Duration.ofSeconds(10))  
    .setBackoffCoefficient(1.2)  
    .setMaximumInterval(Duration.ofSeconds(10))  
    .build()
```

```
var prevPolicy = RetryOptions.newBuilder()  
    .setInitialInterval(Duration.ofSeconds(1))  
    .setMaxAttempts(2)  
    .build()
```

주문 API 호출
재시도 전략

Agent 호출
재시도 전략

과거 보상 API
호출 재시도 전략

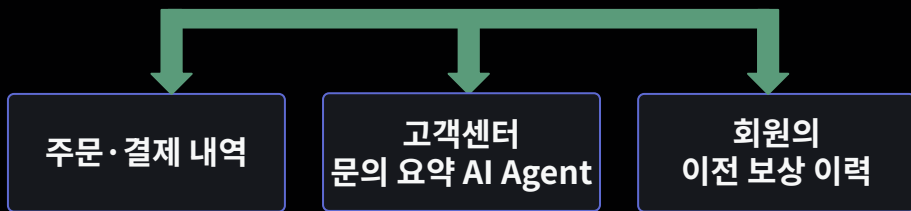
Temporal - Workflow

데이터 조회 부분 코드

```
// FAN OUT
var orderPromise = Async.function(
    () -> orderAct.fetchOrder(orderId))
var agentPromise = Async.function(
    () -> agentAct.summary(customerId));
var prevPromise = Async.function(
    () -> prevAct.fetchPrev(customerId));

// 5분 수행 시간 제한
try {
    Promise.all([orderPromise, agentPromise, prevPromise])
        .get(5, TimeUnit.MINUTES);
} catch (TimeoutException e) {
    // catch문 밖에서 실패 평가
}
```

FAN OUT



Temporal - Workflow

데이터 조회 부분 코드

```
// FAN OUT
var orderPromise = Async.function(
    () -> orderAct.fetchOrder(orderId))
var agentPromise = Async.function(
    () -> agentAct.summary(customerId));
var prevPromise = Async.function(
    () -> prevAct.fetchPrev(customerId));

// 5분 수행 시간 제한
try {
    Promise.all([orderPromise, agentPromise, prevPromise])
        .get(5, TimeUnit.MINUTES);
} catch (TimeoutException e) {
    // catch문 밖에서 실패 평가
}
```

시간 내 수행된 결과만 사용

주문·결제 내역

고객센터
문의 요약 AI Agent

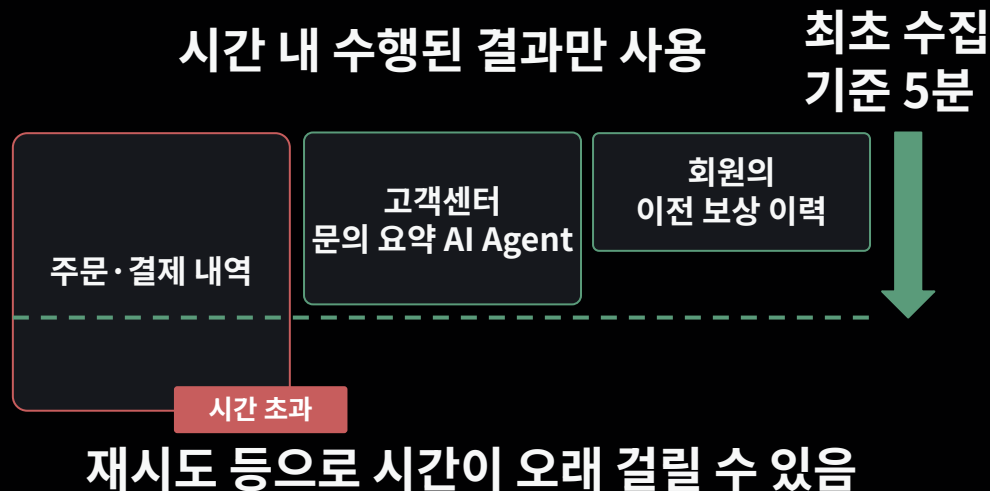
회원의
이전 보상 이력

Temporal - Workflow

데이터 조회 부분 코드

```
// FAN OUT
var orderPromise = Async.function(
  () -> orderAct.fetchOrder(orderId))
var agentPromise = Async.function(
  () -> agentAct.summary(customerId));
var prevPromise = Async.function(
  () -> prevAct.fetchPrev(customerId));

// 5분 수행 시간 제한
try {
  Promise.all([orderPromise, agentPromise, prevPromise])
    .get(5, TimeUnit.MINUTES);
} catch (TimeoutException e) {
  // catch문 밖에서 실패 평가
}
```

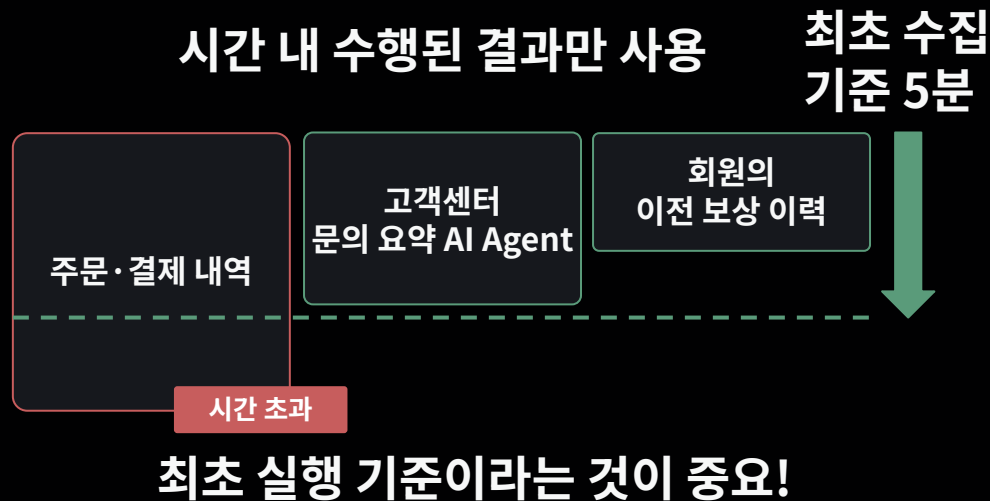


Temporal - Workflow

데이터 조회 부분 코드

```
// FAN OUT
var orderPromise = Async.function(
    () -> orderAct.fetchOrder(orderId))
var agentPromise = Async.function(
    () -> agentAct.summary(customerId));
var prevPromise = Async.function(
    () -> prevAct.fetchPrev(customerId));

// 5분 수행 시간 제한
try {
    Promise.all([orderPromise, agentPromise, prevPromise])
        .get(5, TimeUnit.MINUTES);
} catch (TimeoutException e) {
    // catch문 밖에서 실패 평가
}
```



Temporal - Workflow

데이터 조회 부분 코드

```
// FAN OUT
var orderPromise = Async.function(
    () -> orderAct.fetchOrder(orderId))
var agentPromise = Async.function(
    () -> agentAct.summary(customerId));
var prevPromise = Async.function(
    () -> prevAct.fetchPrev(customerId));

// 5분 수행 시간 제한
try {
    Promise.all([orderPromise, agentPromise, prevPromise])
        .get(5, TimeUnit.MINUTES);
} catch (TimeoutException e) {
    // catch문 밖에서 실패 평가
}
```

timeout

결과영속화

Temporal - Workflow

금액 평가 및 매니저 승인 대기 코드

```
var amount = calcCompensationAmount(context)
managerAct.notifyToManager(amount)

// 최초 수행으로 부터 7일간 승인 대기
var approvalTime = Workflow.await(Duration.days(7),
    () -> approve)

// 최초 수행으로부터 7일 이내 승인 실패
if (!approvalTime) {
    return false
}
```

```
@SignalMethod
void approve() {
    approve = true
}
```

Temporal - Workflow

금액 평가 및 매니저 승인 대기 코드

```
var amount = calcCompensationAmount(context)
managerAct.notifyToManager(amount)

// 최초 수행으로 부터 7일간 승인 대기
var approvalTime = Workflow.await(Duration.days(7),
    () -> approve)

// 최초 수행으로부터 7일 이내 승인 실패
if (!approvalTime) {
    return false
}
```

```
@SignalMethod
void approve() {
    approve = true
}
```

← 금액 평가 및 매니저에게 알림

Temporal - Workflow

금액 평가 및 매니저 승인 대기 코드

```
var amount = calcCompensationAmount(context)
managerAct.notifyToManager(amount)

// 최초 수행으로 부터 7일간 승인 대기
var approvalInTime = Workflow.await(Duration.days(7),
    () -> approve)

// 최초 수행으로부터 7일 이내 승인 실패
if (!approvalInTime) {
    return false
}
```

```
@SignalMethod
void approve() {
    approve = true
}
```

최초 수행으로부터
7일간 승인 대기 시작



Temporal - Workflow

금액 평가 및 매니저 승인 대기 코드

```
var amount = calcCompensationAmount(context)
managerAct.notifyToManager(amount)

// 최초 수행으로 부터 7일간 승인 대기
var approvalTime = Workflow.await(Duration.days(7),
    () -> approve)

// 최초 수행으로 부터 7일 이내 승인 실패
if (!approvalTime) {
    return false
}
```

최초 수행으로부터
7일간 승인 대기 시작

```
@SignalMethod
void approve() {
    approve = true
}
```

매니저가 승인하면
이 함수를 호출하게 만듦

Temporal - Workflow

금액 평가 및 매니저 승인 대기 코드

```
var amount = calcCompensationAmount(context)
managerAct.notifyToManager(amount)

// 최초 수행으로부터 7일간 승인 대기
var approvalTime = Workflow.await(Duration.days(7),
    () -> approve)

// 최초 수행으로부터 7일 이내 승인 실패
if (!approvalTime) {
    return false
}
```

최초 수행으로부터
7일간 승인 대기 시작

approve가 True가 되면
즉시 깨어남

```
@SignalMethod
void approve() {
    approve = true
}
```

매니저가 승인하면
이 함수를 호출하게 만듦

Temporal - Workflow

금액 평가 및 매니저 승인 대기 코드

```
var amount = calcCompensationAmount(context)
managerAct.notifyToManager(amount)

// 최초 수행으로 부터 7일간 승인 대기
var approvalTime = Workflow.await(Duration.days(7),
    () -> approve)

// 최초 수행으로부터 7일 이내 승인 실패
if (!approvalTime) {
    return false
}
```

```
@SignalMethod
void approve() {
    approve = true
}
```

타이머

신호처리

Temporal - Workflow

지급 및 실패 시 보상 코드

```
var saga = new Saga(new Saga.Options.Builder()
    .setParallelCompensation(false)
    .build())

try {
    // 지급 준비 기록
    saga.addCompensation(() -> payAct.cancelPre(txId)
    var prepareRes = payAct.prepareCredit(customerId, amount, txId)
    // 고객 지급
    saga.addCompensation(() -> transAct.refund(txId));
    var receipt = transAct.payCredit(customerId, amount, txId)
    // 지급 완료 기록
    payAct.confirmCredit(customerId, prepareRes, receipt, txId)
    return true
} catch (Exception e) {
    // 보상 수행
    saga.compensate()
    return false
}
```

회계 장부
지급 준비 기록



고객 지급



회계 장부
지급 완료 기록

Temporal - Workflow

지급 및 실패 시 보상 코드

```
var saga = new Saga(new Saga.Options.Builder()
    .setParallelCompensation(false)
    .build())

try {
    // 지급 준비 기록
    saga.addCompensation(() -> payAct.cancelPre(txId)
    var prepareRes = payAct.prepareCredit(customerId, amount, txId)
    // 고객 지급
    saga.addCompensation(() -> transAct.refund(txId));
    var receipt = transAct.payCredit(customerId, amount, txId)
    // 지급 완료 기록
    payAct.confirmCredit(customerId, prepareRes, receipt, txId)
    return true
} catch (Exception e) {
    // 보상 수행
    saga.compensate()
    return false
}
```

회계 장부
지급 준비 기록



고객 지급



회계 장부
지급 완료 기록

Temporal - Workflow

지급 및 실패 시 보상 코드

```
var saga = new Saga(new Saga.Options.Builder()
    .setParallelCompensation(false)
    .build())

try {
    // 지급 준비 기록
    saga.addCompensation(() -> payAct.cancelPre(txId)
    var prepareRes = payAct.prepareCredit(customerId, amount, txId)
    // 고객 지급
    saga.addCompensation(() -> transAct.refund(txId));
    var receipt = transAct.payCredit(customerId, amount, txId)
    // 지급 완료 기록
    payAct.confirmCredit(customerId, prepareRes, receipt, txId)
    return true
} catch (Exception e) {
    // 보상 수행
    saga.compensate()
    return false
}
```

회계 장부
지급 준비 기록



고객 지급



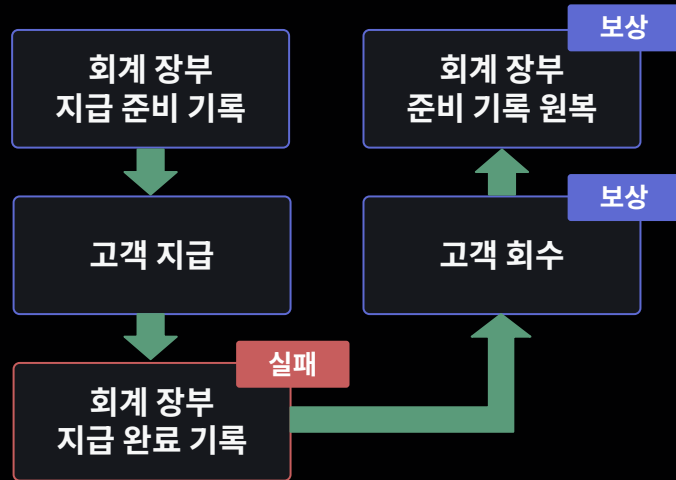
회계 장부
지급 완료 기록

Temporal - Workflow

지급 및 실패 시 보상 코드

```
var saga = new Saga(new Saga.Options.Builder()
    .setParallelCompensation(false)
    .build())

try {
    // 지급 준비 기록
    saga.addCompensation(() -> payAct.cancelPre(txId)
    var prepareRes = payAct.prepareCredit(customerId, amount, txId)
    // 고객 지급
    saga.addCompensation(() -> transAct.refund(txId));
    var receipt = transAct.payCredit(customerId, amount, txId)
    // 지급 완료 기록
    payAct.confirmCredit(customerId, prepareRes, receipt, txId)
    return true
} catch (Exception e) {
    // 보상 수행
    saga.compensate()
    return false
}
```



Temporal - Workflow

지급 및 실패 시 보상 코드

```
var saga = new Saga(new Saga.Options.Builder()
    .setParallelCompensation(false)
    .build())

try {
    // 지급 준비 기록
    saga.addCompensation(() -> payAct.cancelPre(txId)
    var prepareRes = payAct.prepareCredit(customerId, amount, txId)
    // 고객 지급
    saga.addCompensation(() -> transAct.refund(txId));
    var receipt = transAct.payCredit(customerId, amount, txId)
    // 지급 완료 기록
    payAct.confirmCredit(customerId, prepareRes, receipt, txId)
    return true
} catch (Exception e) {
    // 보상 수행
    saga.compensate()
    return false
}
```

All or Nothing

보상

```
class Saga {
```

```
...
```

```
}
```

```
class Task {
```

```
...
```

```
}
```

```
class DistributedLock {
```

```
...
```

```
}
```

```
class TaskContext {
```

```
...
```

```
}
```

```
class TaskFactory {
```

```
...
```

```
}
```

```
class TaskScheduler {
```

```
...
```

```
}
```

```
class TaskReceptor {
```

```
...
```

```
}
```

```
class TaskWatcher {
```

```
...
```

```
}
```

```
void process(Ticket ticket) {  
    // 수집  
    var context = collectDataConcurrently(ticket)  
    // 평가  
    var amount = evaluate(context)  
    // 7일 대기 혹은 매니저 승인  
    await(Duration.ofDays(7), () -> approved)  
    if (approved) {  
        // 지급  
        recordAndPay(amount, ticket)  
    }  
}
```

```
class AsyncExecutor {
```

```
...
```

```
}
```

```
class Compensation {
```

```
...
```

```
}
```

```
class TaskExecutor {
```

```
...
```

```
}
```

```
enum Status {
```

```
...
```

```
}
```

```
class TaskContextRepository {
```

```
...
```

목차

01

워크플로우는 왜 항상 복잡할까

고객 보상 시나리오 - 비즈니스 코드가 파묻히는 과정

02

Temporal이 제안하는 방식

실패가 없는 것처럼 짜고 나머지는 Temporal이 맡는다

03

어떻게 가능한가? (Durable Execution의 원리)

Event History · Replay · Proxy — 멈췄다 그 자리에서 이어지는 원리

04

그래서 무엇이 남았나

비즈니스 시맨틱만 남은 코드와 복잡도의 행방

**경고: 지금부터는 단순화한
개념모형입니다
실제 엔진은 훨씬 정교합니다**

핵심 요소

4가지 요소

Workflow
코드

Event
History

Replay
(Event Sourcing)

Proxy

핵심 요소

4가지 요소

Workflow
코드

Event
History

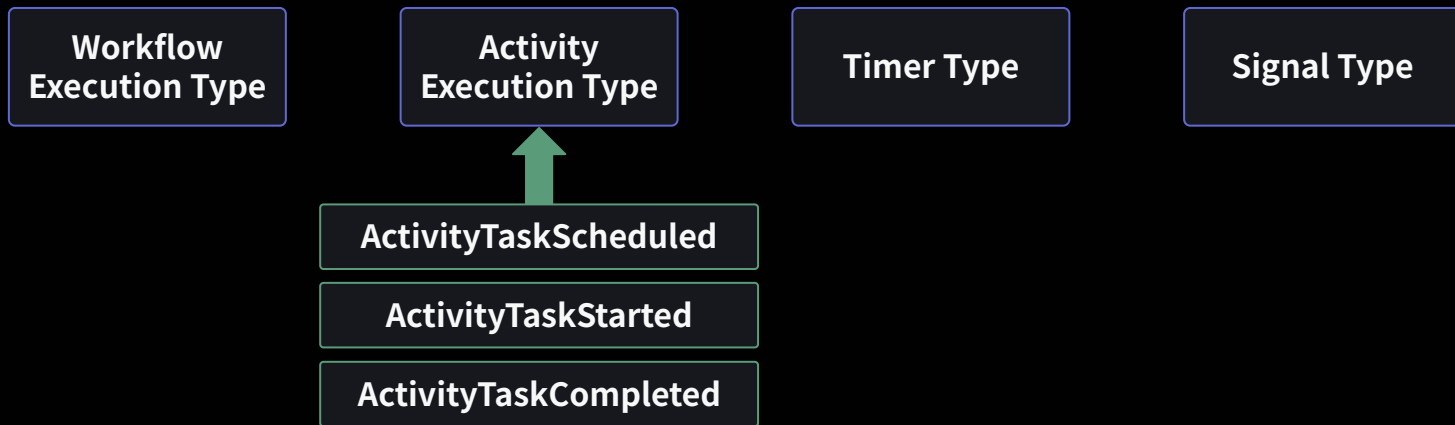
Replay
(Event Sourcing)

Proxy

Event History



Event History



핵심 요소

Replay

중단
&
1일후 재개

```
void process(MyCmd command) {  
  var a = actA.foo(command.aArg)  
  
  Workflow.sleep(Duration.ofDays(1))  
  
  actB.bar(a)  
}
```

JVM Stack

Process

함수 Stack 영속화
&
1일 후 재실행
불가능

핵심 요소

Replay

중단
&
1일후 재개

```
void process(MyCmd command) {  
    var a = actA.foo(command.aArg)  
    Workflow.sleep(Duration.ofDays(1))  
    actB.bar(a)  
}
```

JVM Stack

Process

함수 Stack 영속화
&
1일 후 재실행
불가능

핵심 요소

Replay

```
class MyWorkflow implements MyWorkflowInterface {  
    void process(MyCommand command) {  
        var a = actA.foo(command.aArg)  
  
        Workflow.sleep(Duration.ofDays(1))  
  
        actB.bar(a)  
    }  
}
```

Event History (Temporal Server)

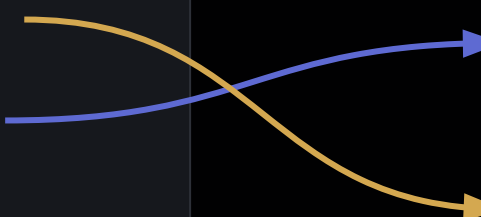
Timer Fired

⋮

Activity A Completed(a)

⋮

Workflow Started



핵심 요소

Replay

```
class MyWorkflow implements MyWorkflowInterface {  
    void process(MyCommand command) {  
        var a = actA.foo(command.aArg)  
  
        Workflow.sleep(Duration.ofDays(1))  
  
        actB.bar(a)  
    }  
}
```

누가 분기?

Event History (Temporal Server)

Timer Fired

⋮

Activity A Completed(a)

⋮

Workflow Started

핵심 요소

Proxy

```
void process(MyCommand command) {  
    var a = actA.foo(command.aArg)  
  
    var b = actB.bar(command.bArg)  
}
```



Proxy



Event History (Temporal Server)

Activity A Completed(a)

Workflow Started

**함수가 종료되어도
종료된 지점부터 재개되는 것처럼 동작**

MyWorkflow로 보는 Durable Execution

```
public class MyWorkflow implements MyWorkflowInterface {  
  
    @Override  
    public void process(MyCommand command) {  
        var a = actA.foo(command.aArg)  
  
        Workflow.sleep(Duration.ofDays(1))  
  
        actB.bar(a)  
    }  
}
```

실제 동작은 보다 정교하니, 꼭 공식문서와 코드를 같이 참고해주세요

MyWorkflow로 보는 Durable Execution

```
void process(MyCommand command) {  
    var a = actA.foo(command.aArg)  
  
    Workflow.sleep(Duration.ofDays(1))  
  
    actB.bar(a)  
}
```

Event History (Temporal Server)

Task Queue

MyWorkflow로 보는 Durable Execution

```
void process(MyCommand command) {  
    var a = actA.foo(command.aArg)  
  
    Workflow.sleep(Duration.ofDays(1))  
  
    actB.bar(a)  
}
```

Event History (Temporal Server)

Workflow Started

Task Queue

MyWorkflow로 보는 Durable Execution

```
void process(MyCommand command) {  
    var a = actA.foo(command.aArg)  
  
    Workflow.sleep(Duration.ofDays(1))  
  
    actB.bar(a)  
}
```

Event History (Temporal Server)

Workflow Started

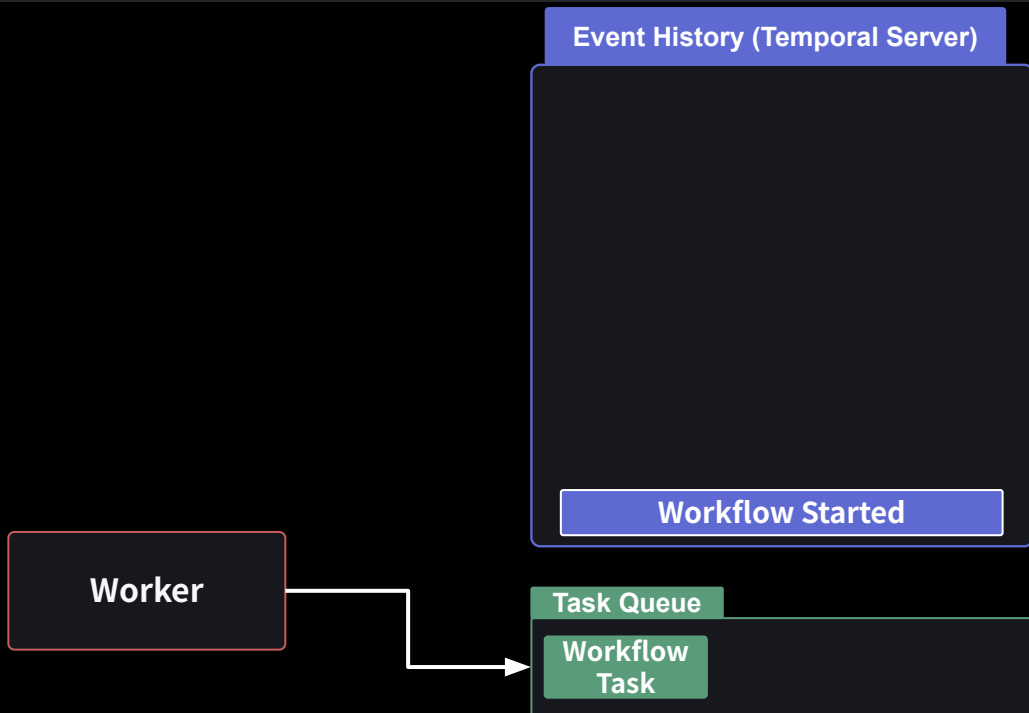


Task Queue

Workflow
Task

MyWorkflow로 보는 Durable Execution

```
void process(MyCommand command) {  
    var a = actA.foo(command.aArg)  
  
    Workflow.sleep(Duration.ofDays(1))  
  
    actB.bar(a)  
}
```



MyWorkflow로 보는 Durable Execution

```
void process(MyCommand command) {  
    var a = actA.foo(command.aArg)  
  
    Workflow.sleep(Duration.ofDays(1))  
  
    actB.bar(a)  
}
```

Worker

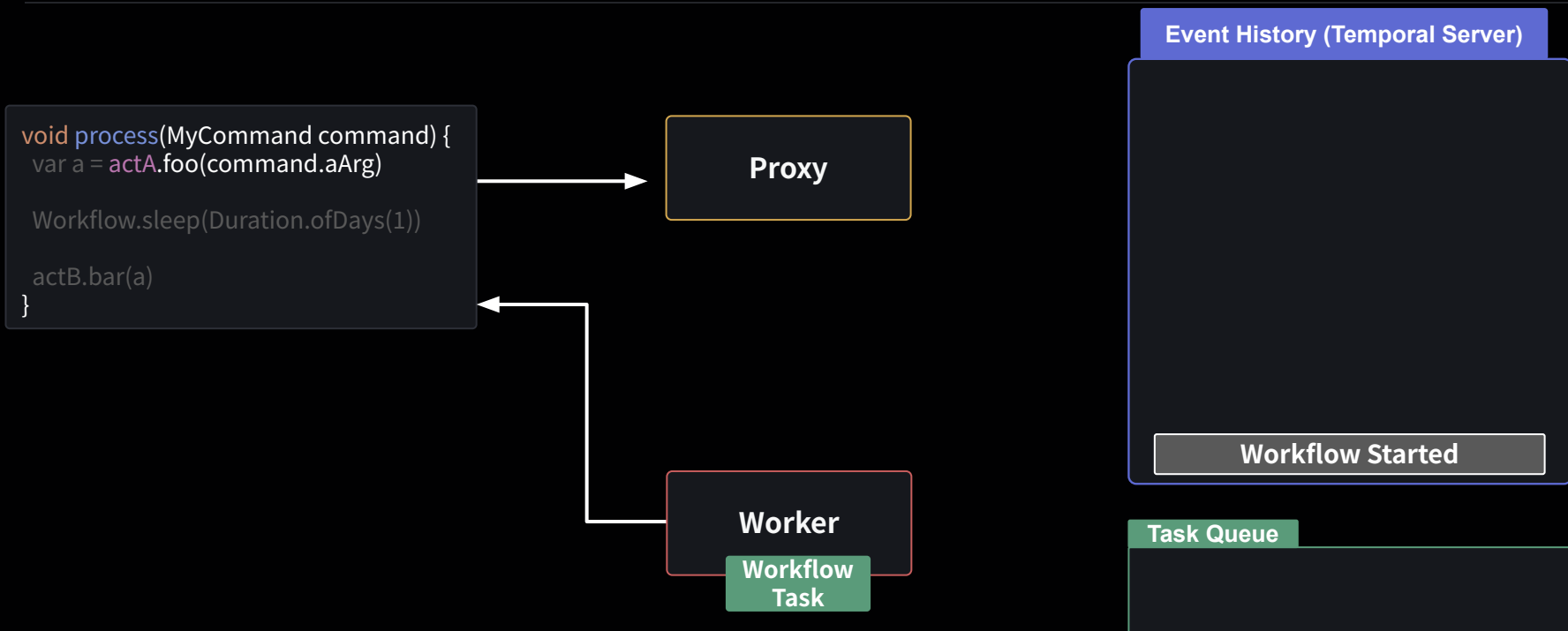
Workflow
Task

Event History (Temporal Server)

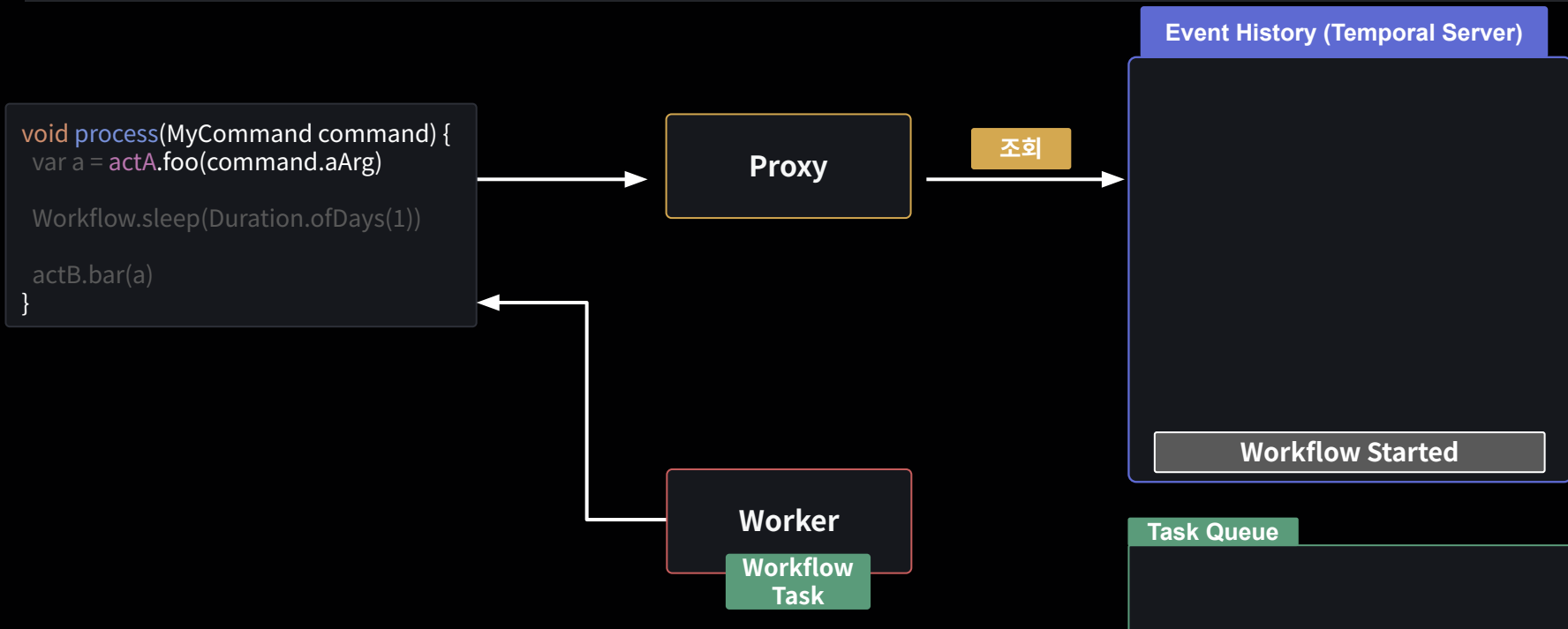
Workflow Started

Task Queue

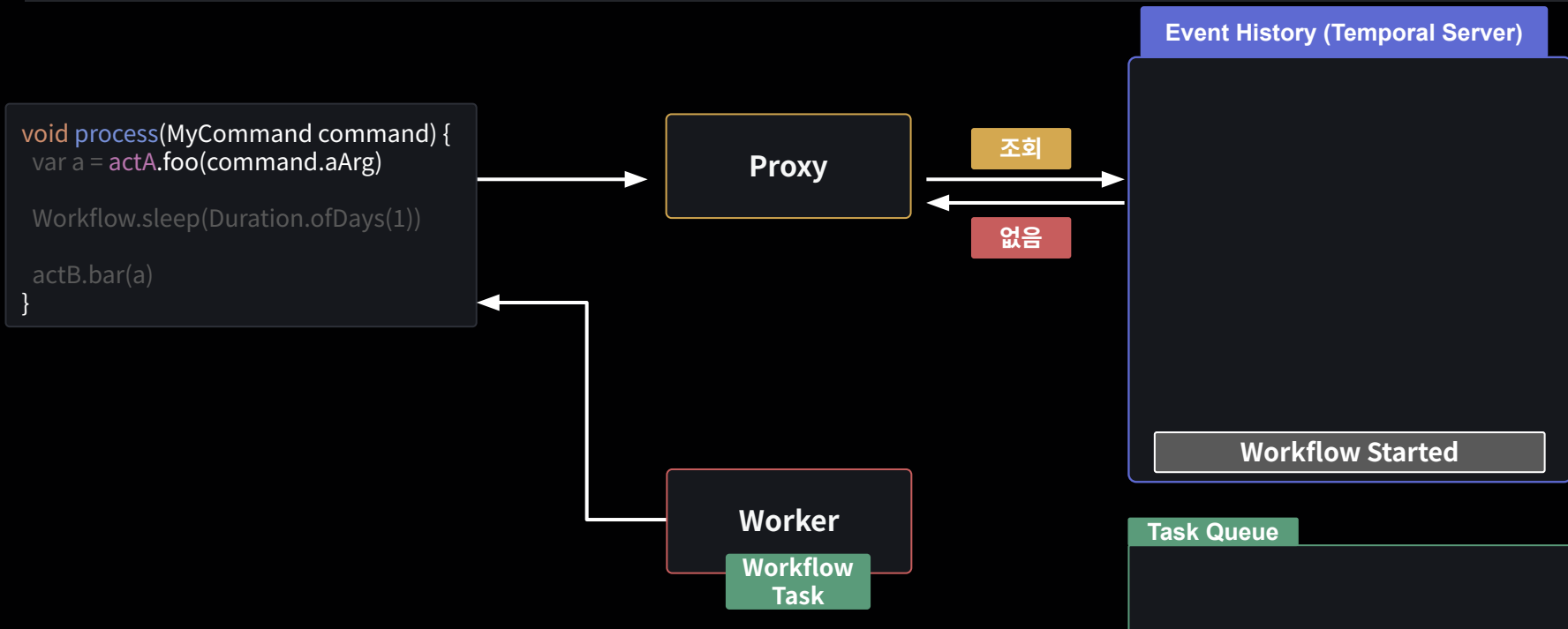
MyWorkflow로 보는 Durable Execution



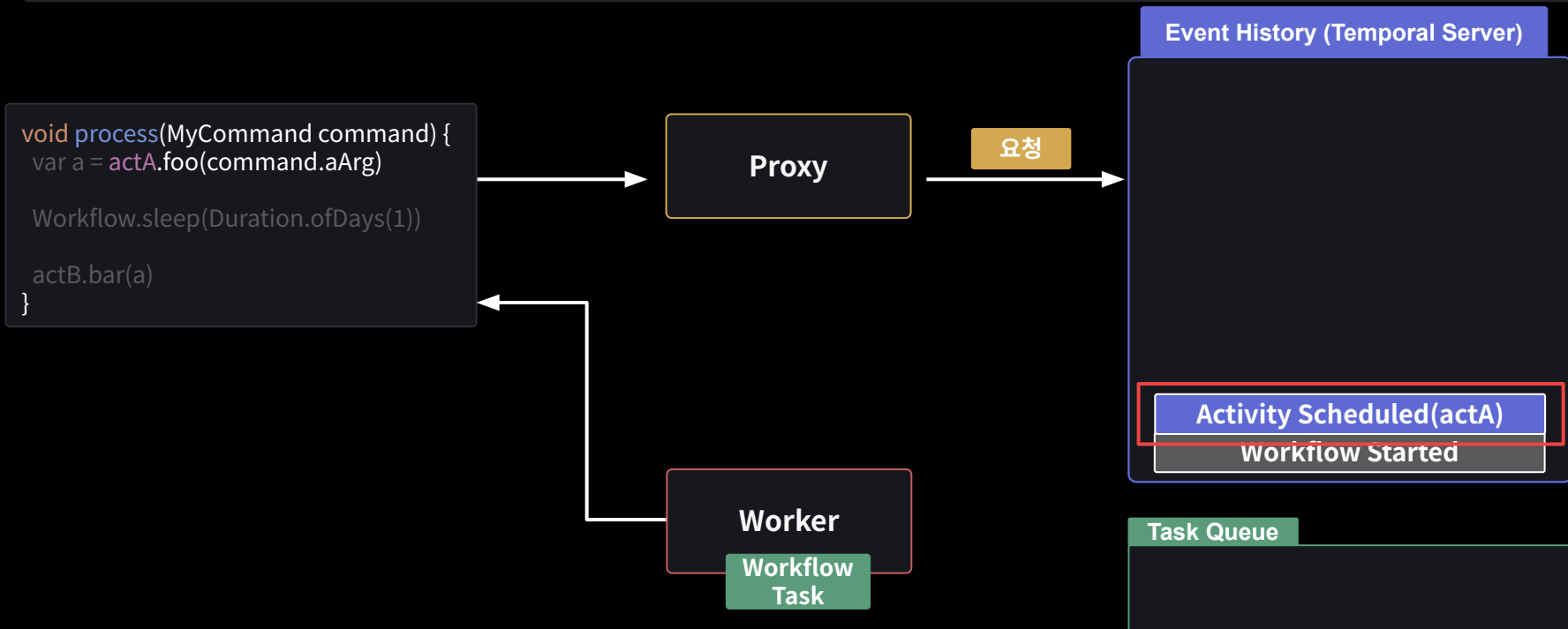
MyWorkflow로 보는 Durable Execution



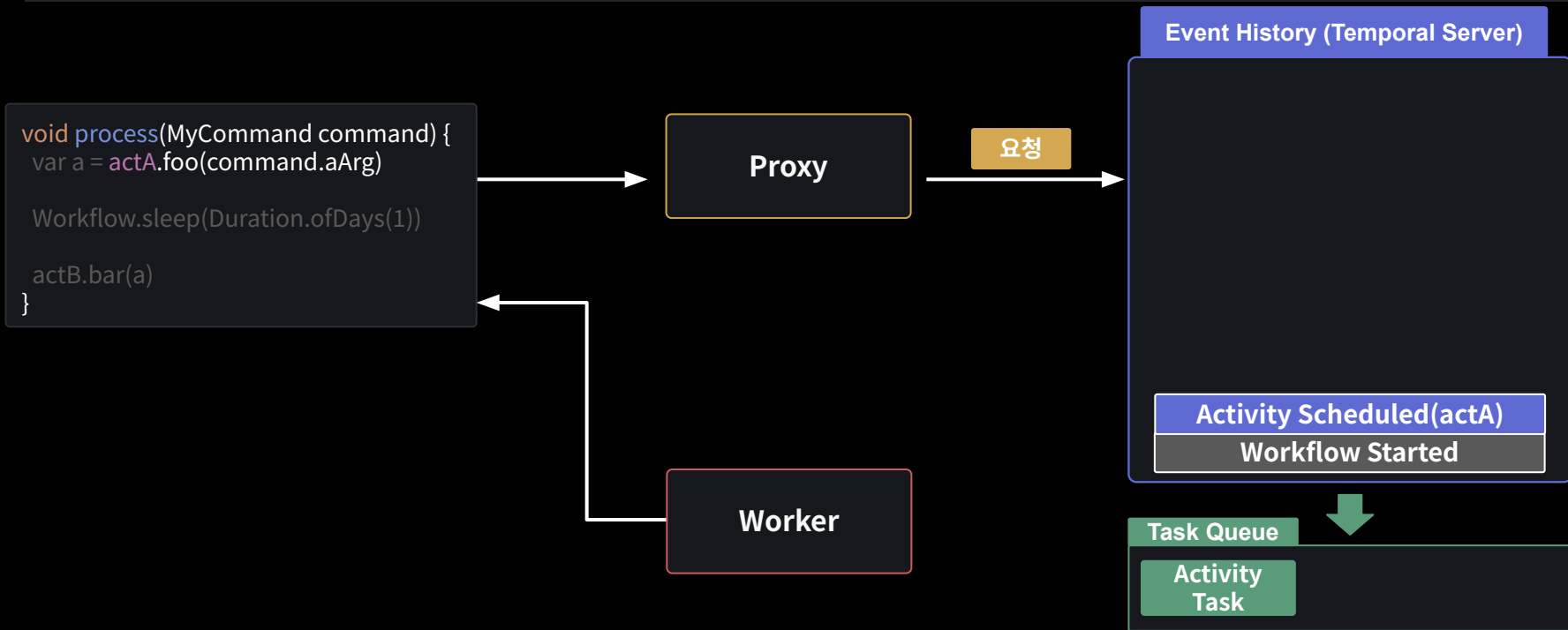
MyWorkflow로 보는 Durable Execution



MyWorkflow로 보는 Durable Execution



MyWorkflow로 보는 Durable Execution



MyWorkflow로 보는 Durable Execution

```
void process(MyCommand command) {  
    var a = actA.foo(command.aArg)  
  
    Workflow.sleep(Duration.ofDays(1))  
  
    actB.bar(a)  
}
```

Proxy

Worker

Event History (Temporal Server)

Activity Started(actA)

Activity Scheduled(actA)

Workflow Started

Task Queue

Activity
Task

MyWorkflow로 보는 Durable Execution

```
void process(MyCommand command) {  
    var a = actA.foo(command.aArg)  
  
    Workflow.sleep(Duration.ofDays(1))  
  
    actB.bar(a)  
}
```

Proxy

Activity

수행

Worker

Activity
Task

Event History (Temporal Server)

Activity Started(actA)

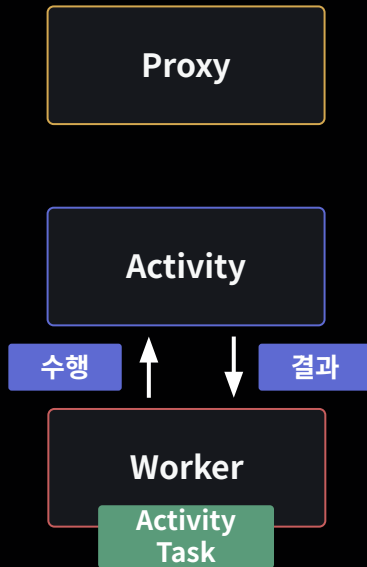
Activity Scheduled(actA)

Workflow Started

Task Queue

MyWorkflow로 보는 Durable Execution

```
void process(MyCommand command) {  
    var a = actA.foo(command.aArg)  
  
    Workflow.sleep(Duration.ofDays(1))  
  
    actB.bar(a)  
}
```



Event History (Temporal Server)

Activity Started(actA)

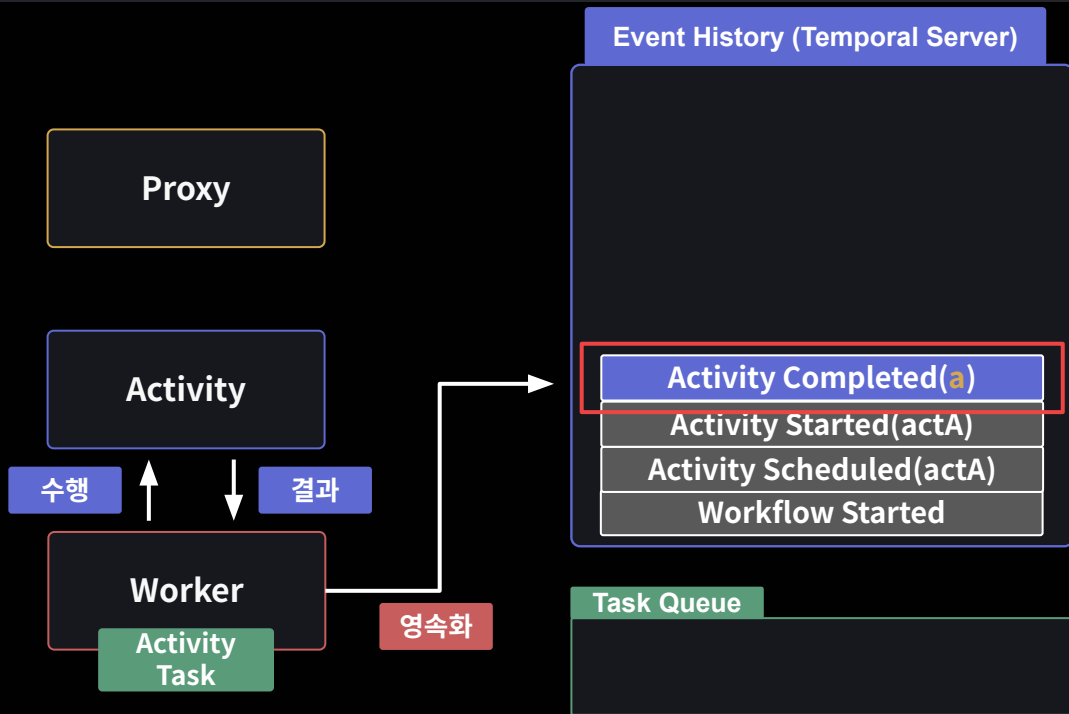
Activity Scheduled(actA)

Workflow Started

Task Queue

MyWorkflow로 보는 Durable Execution

```
void process(MyCommand command) {  
    var a = actA.foo(command.aArg)  
  
    Workflow.sleep(Duration.ofDays(1))  
  
    actB.foo(a)  
}
```



MyWorkflow로 보는 Durable Execution

```
void process(MyCommand command) {  
    var a = actA.foo(command.aArg)  
  
    Workflow.sleep(Duration.ofDays(1))  
  
    actB.bar(a)  
}
```

Proxy

Worker

Event History (Temporal Server)

Activity Completed(a)

Activity Started(actA)

Activity Scheduled(actA)

Workflow Started



Task Queue

Workflow
Task

MyWorkflow로 보는 Durable Execution

```
void process(MyCommand command) {  
    var a = actA.foo(command.aArg)  
  
    Workflow.sleep(Duration.ofDays(1))  
  
    actB.bar(a)  
}
```

Proxy

Worker

Event History (Temporal Server)

Activity Completed(a)

Activity Started(actA)

Activity Scheduled(actA)

Workflow Started

Task Queue

Workflow
Task

MyWorkflow로 보는 Durable Execution

```
void process(MyCommand command) {  
    var a = actA.foo(command.aArg)  
  
    Workflow.sleep(Duration.ofDays(1))  
  
    actB.bar(a)  
}
```

Proxy

Worker

Workflow
Task

Event History (Temporal Server)

Activity Completed(a)

Activity Started(actA)

Activity Scheduled(actA)

Workflow Started

Task Queue

MyWorkflow로 보는 Durable Execution

```
void process(MyCommand command) {  
    var a = actA.foo(command.aArg)  
  
    Workflow.sleep(Duration.ofDays(1))  
  
    actB.bar(a)  
}
```

Proxy

Worker

Workflow
Task

Event History (Temporal Server)

Activity Completed(a)

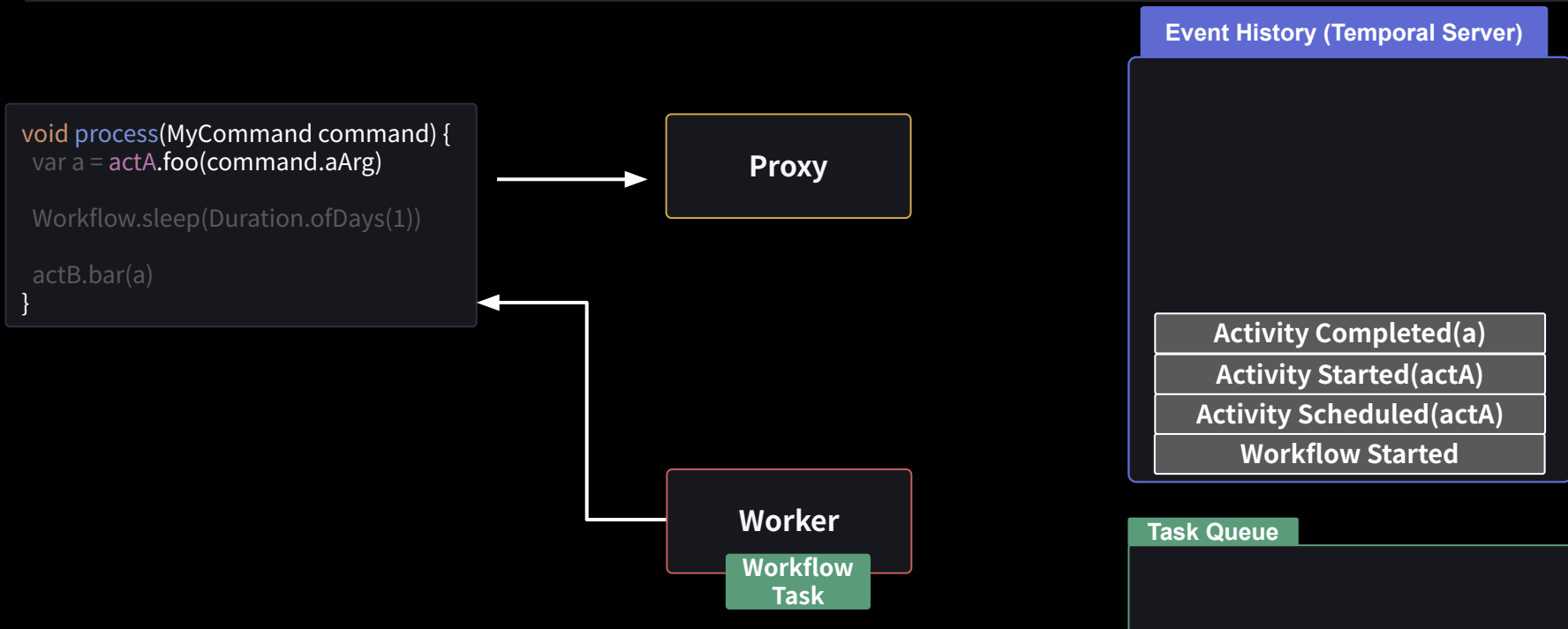
Activity Started(actA)

Activity Scheduled(actA)

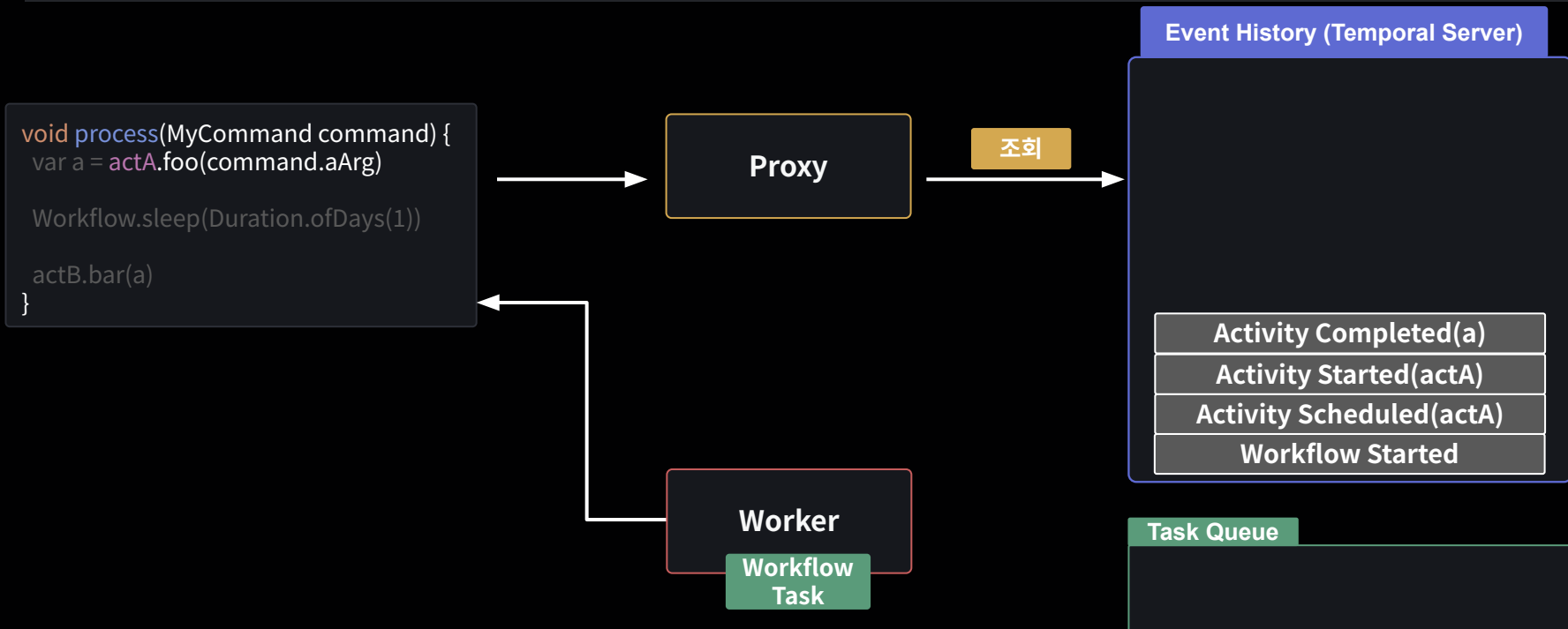
Workflow Started

Task Queue

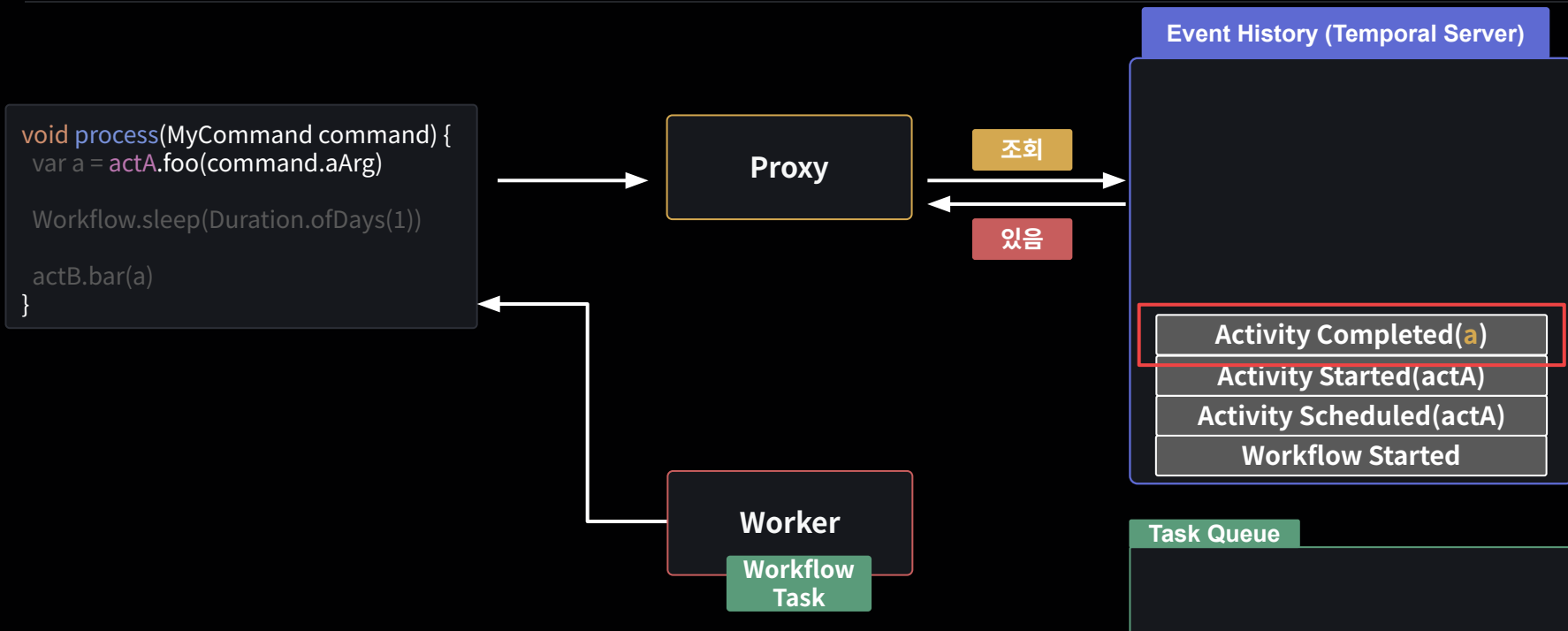
MyWorkflow로 보는 Durable Execution



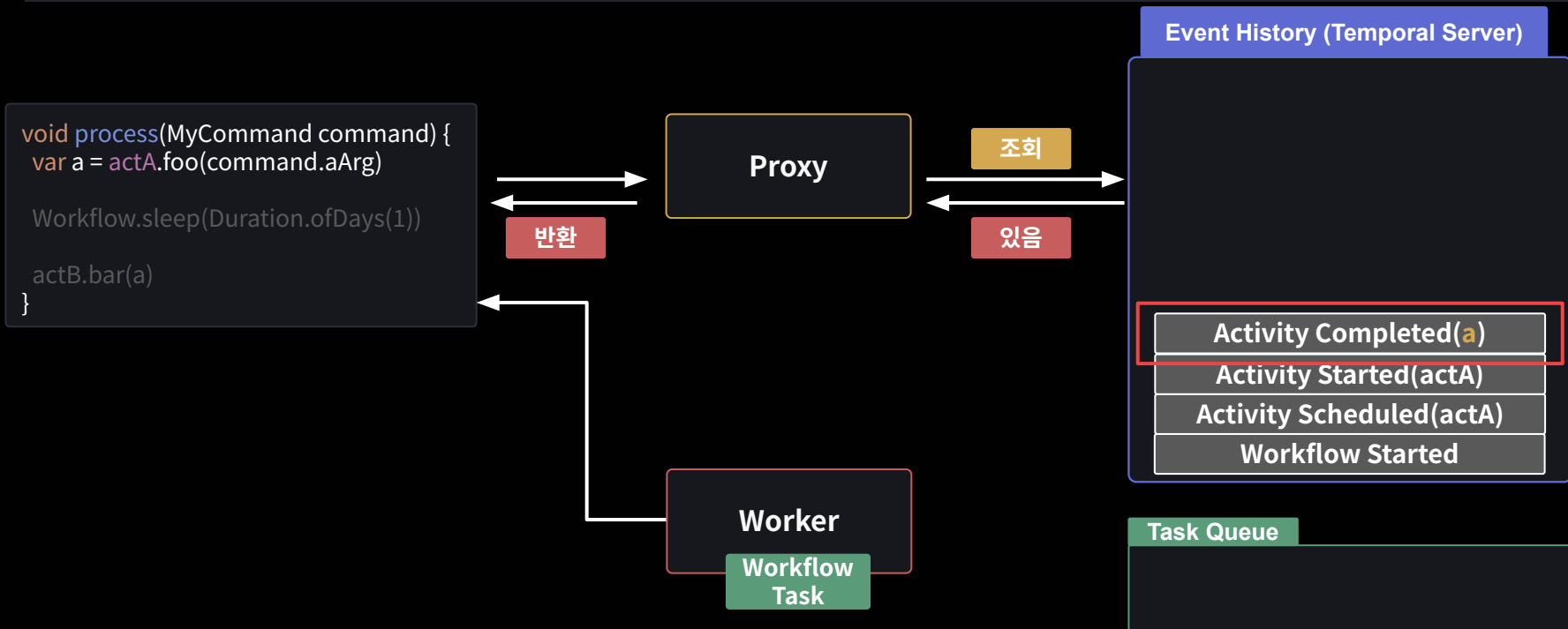
MyWorkflow로 보는 Durable Execution



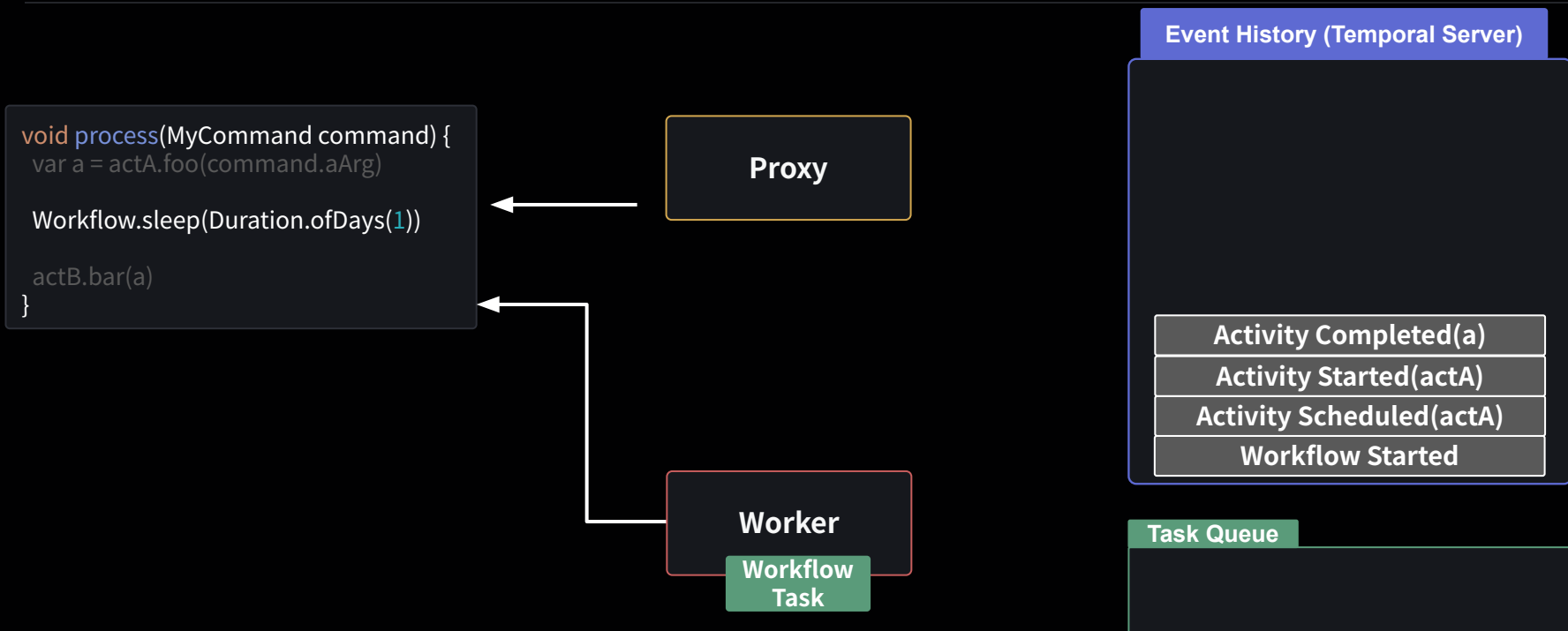
MyWorkflow로 보는 Durable Execution



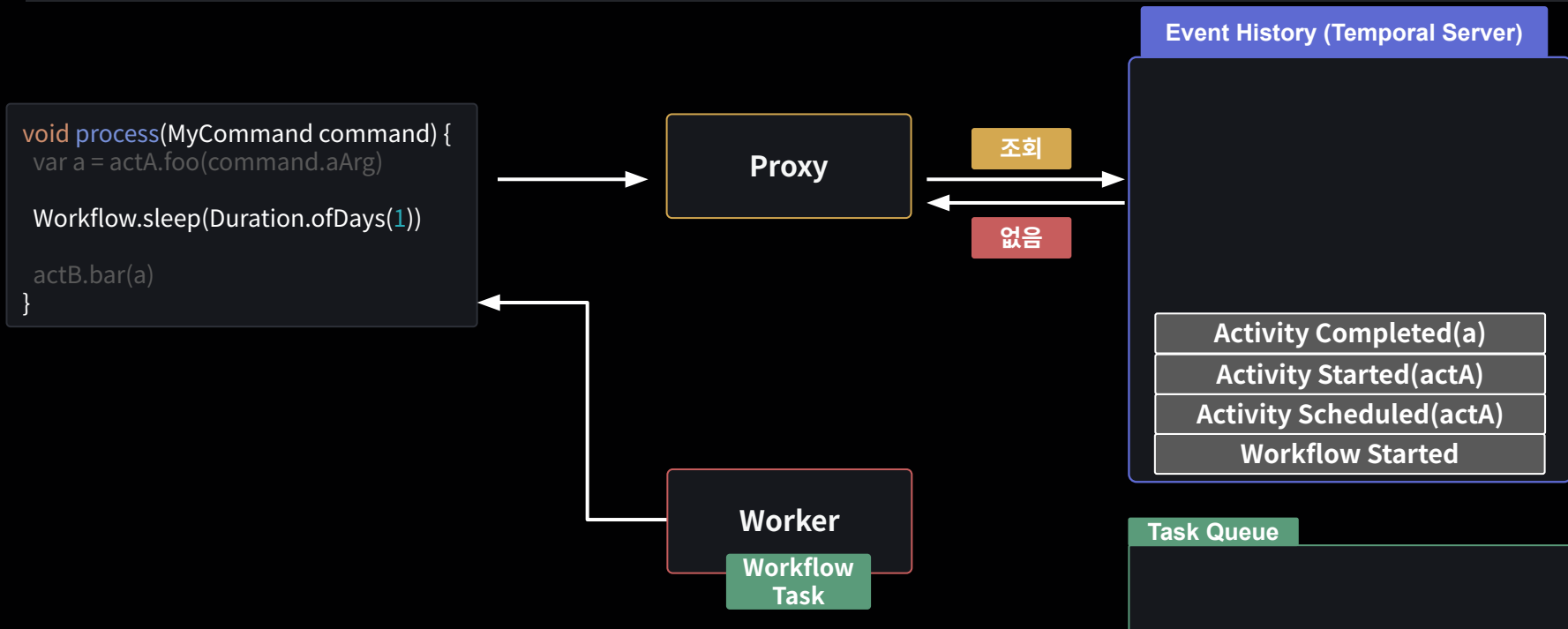
MyWorkflow로 보는 Durable Execution



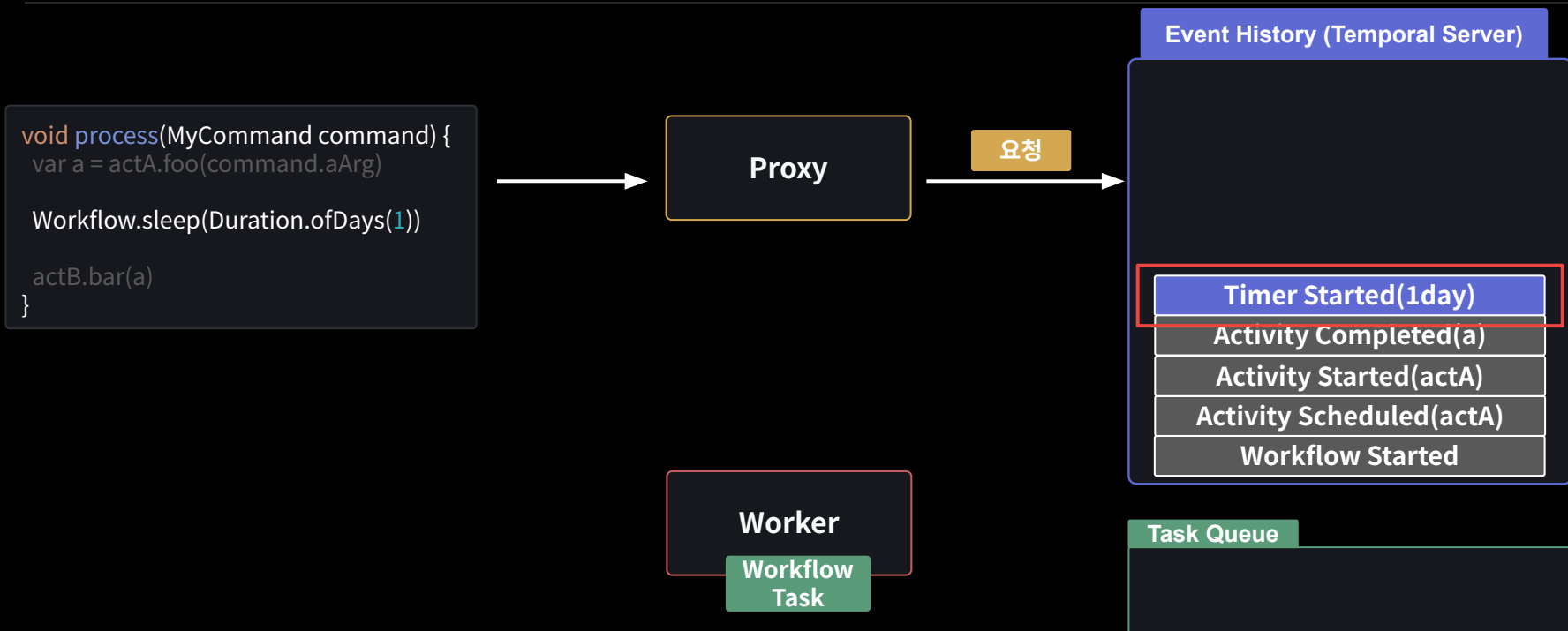
MyWorkflow로 보는 Durable Execution



MyWorkflow로 보는 Durable Execution



MyWorkflow로 보는 Durable Execution



MyWorkflow로 보는 Durable Execution

```
void process(MyCommand command) {  
    var a = actA.foo(command.aArg)  
  
    Workflow.sleep(Duration.ofDays(1))  
  
    actB.bar(a)  
}
```

Proxy



Worker

Event History (Temporal Server)

Timer Started(1day)

Activity Completed(a)

Activity Started(actA)

Activity Scheduled(actA)

Workflow Started

Task Queue

MyWorkflow로 보는 Durable Execution

```
void process(MyCommand command) {  
    var a = actA.foo(command.aArg)  
  
    Workflow.sleep(Duration.ofDays(1))  
  
    actB.bar(a)  
}
```

Proxy

Worker

Event History (Temporal Server)

Timer Fired

Timer Started(1day)

Activity Completed(a)

Activity Started(actA)

Activity Scheduled(actA)

Workflow Started

Task Queue

Workflow
Task

MyWorkflow로 보는 Durable Execution

```
void process(MyCommand command) {  
    var a = actA.foo(command.aArg)  
  
    Workflow.sleep(Duration.ofDays(1))  
  
    actB.bar(a)  
}
```

Proxy

Worker

Event History (Temporal Server)

Timer Fired

Timer Started(1day)

Activity Completed(a)

Activity Started(actA)

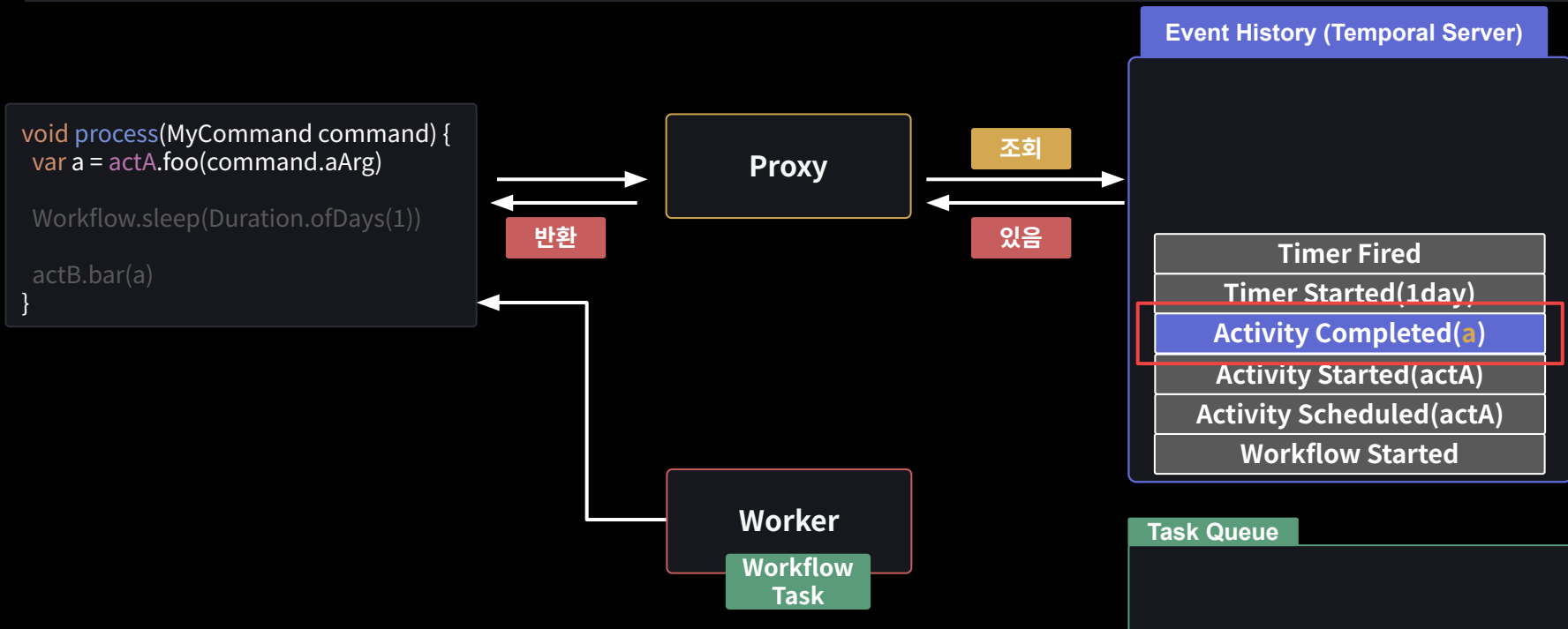
Activity Scheduled(actA)

Workflow Started

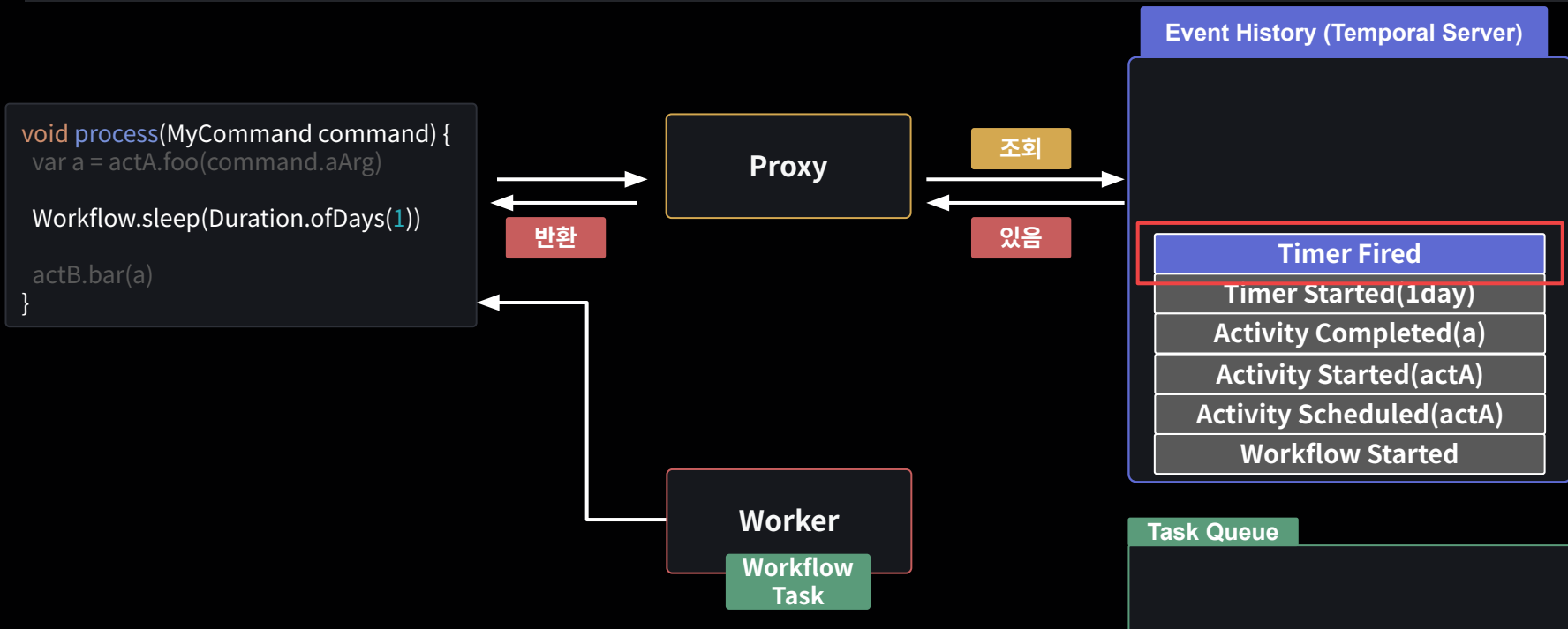
Task Queue

Workflow
Task

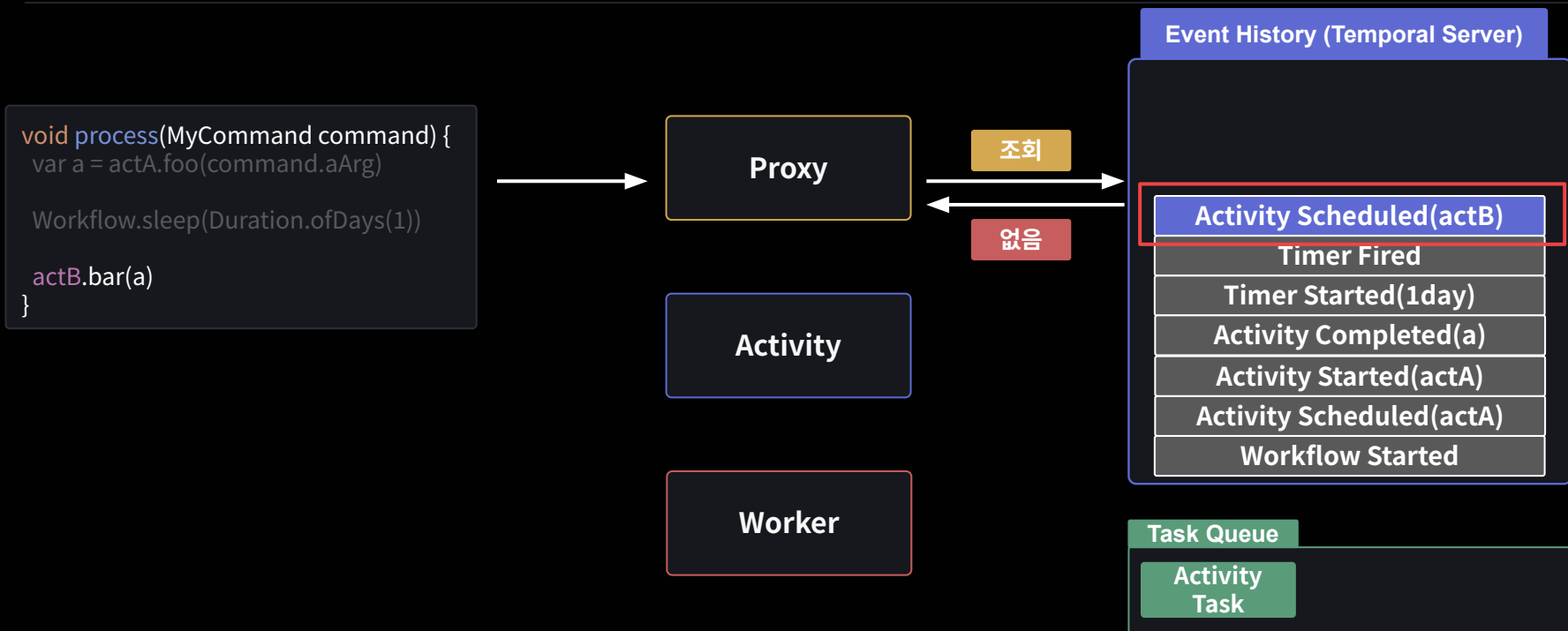
MyWorkflow로 보는 Durable Execution



MyWorkflow로 보는 Durable Execution



MyWorkflow로 보는 Durable Execution



MyWorkflow로 보는 Durable Execution

```
void process(MyCommand command) {  
    var a = actA.foo(command.aArg)  
  
    Workflow.sleep(Duration.ofDays(1))  
  
    actB.bar(a)  
}
```



Proxy

Activity

수행



결과



Worker

Activity
Task

영속화



Event History (Temporal Server)

Activity Completed

Activity Started(actB)

Activity Scheduled(actB)

Timer Fired

Timer Started(1day)

Activity Completed(a)

Activity Started(actA)

Activity Scheduled(actA)

Workflow Started

Task Queue

MyWorkflow로 보는 Durable Execution

```
void process(MyCommand command) {  
    var a = actA.foo(command.aArg)  
  
    Workflow.sleep(Duration.ofDays(1))  
  
    actB.bar(a)  
}
```

Proxy

Activity

Worker

Event History (Temporal Server)

Activity Completed

Activity Started(actB)

Activity Scheduled(actB)

Timer Fired

Timer Started(1day)

Activity Completed(a)

Activity Started(actA)

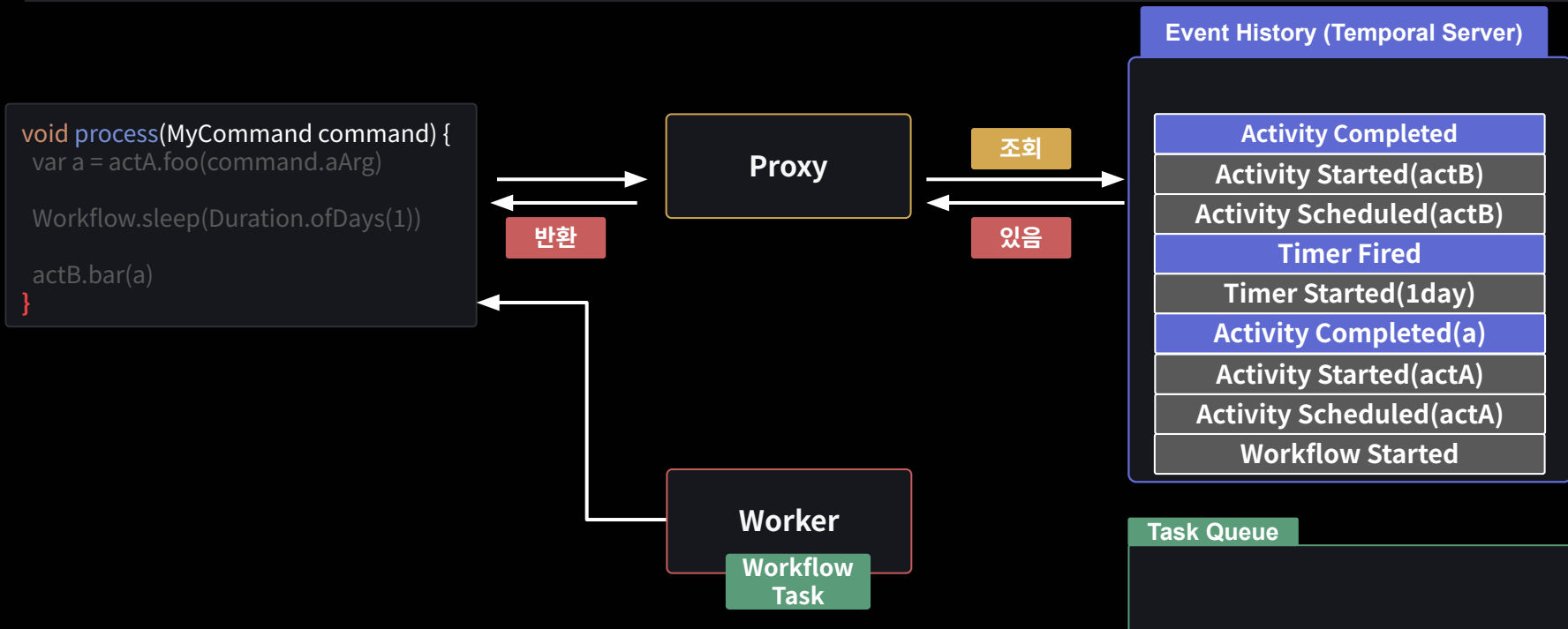
Activity Scheduled(actA)

Workflow Started

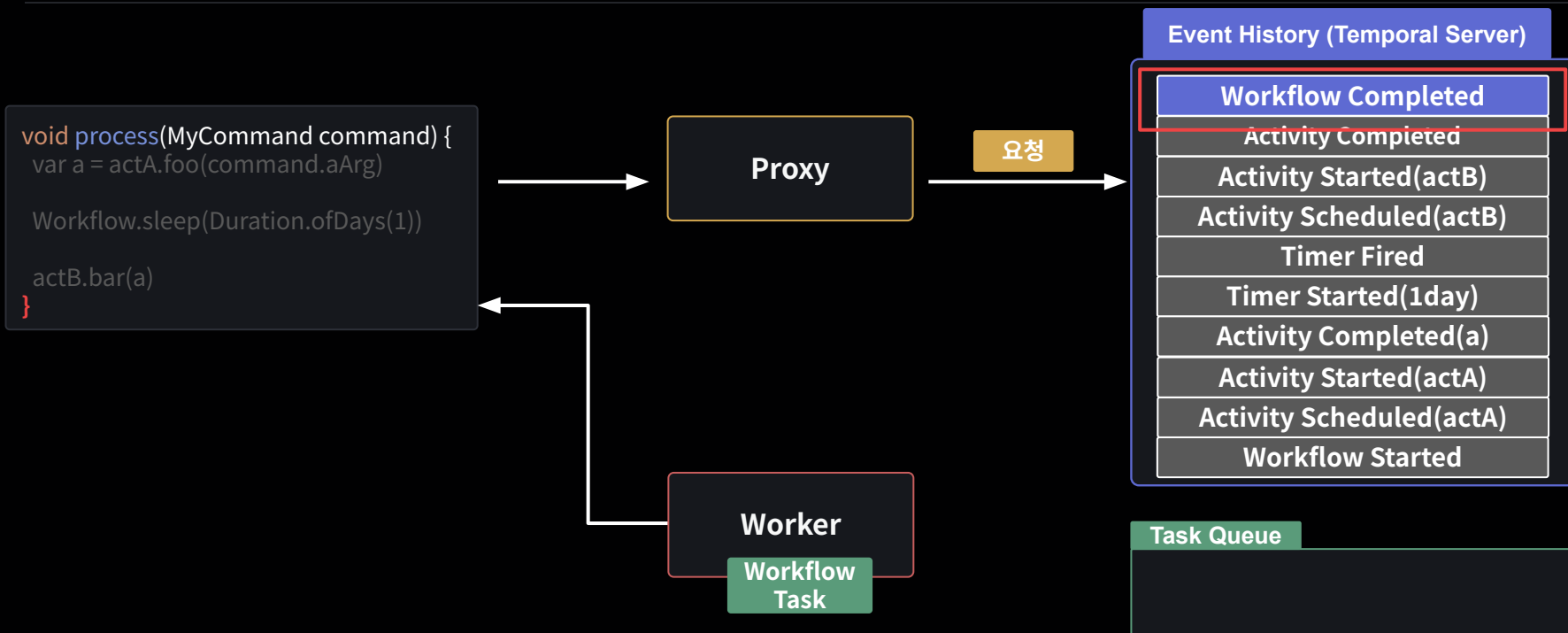
Task Queue

Workflow
Task

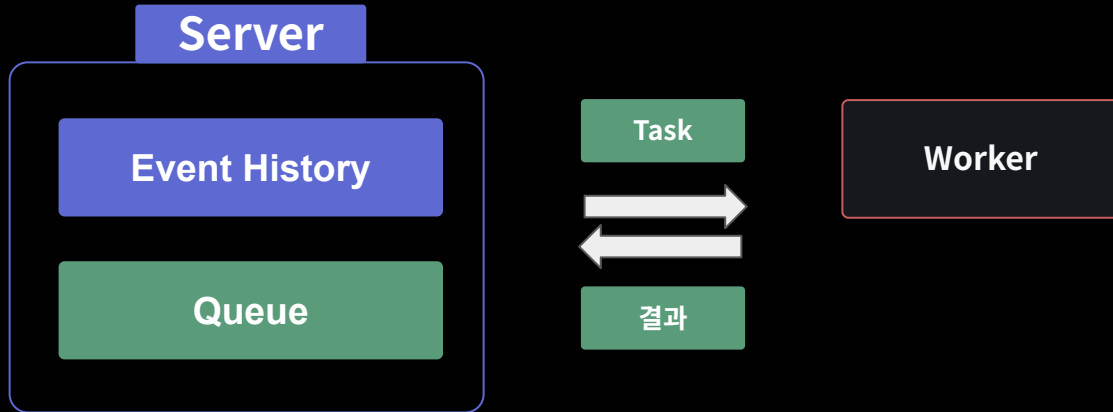
MyWorkflow로 보는 Durable Execution



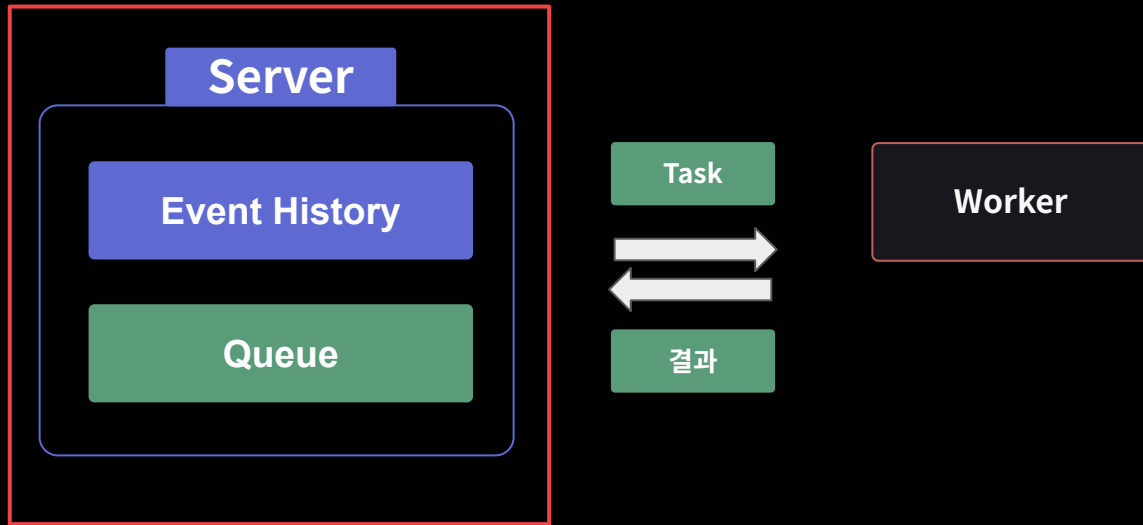
MyWorkflow로 보는 Durable Execution



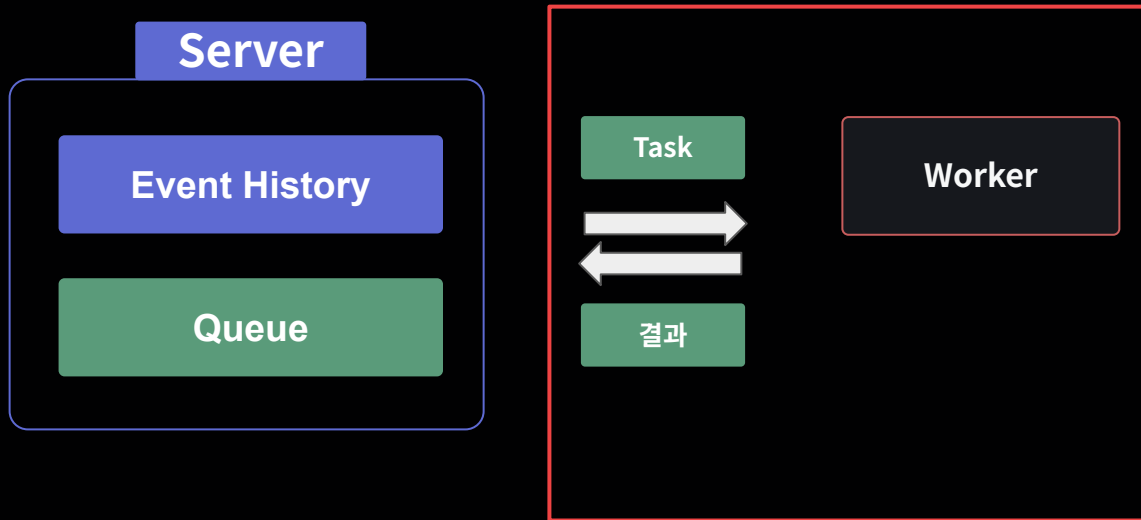
MyWorkflow로 보는 Durable Execution



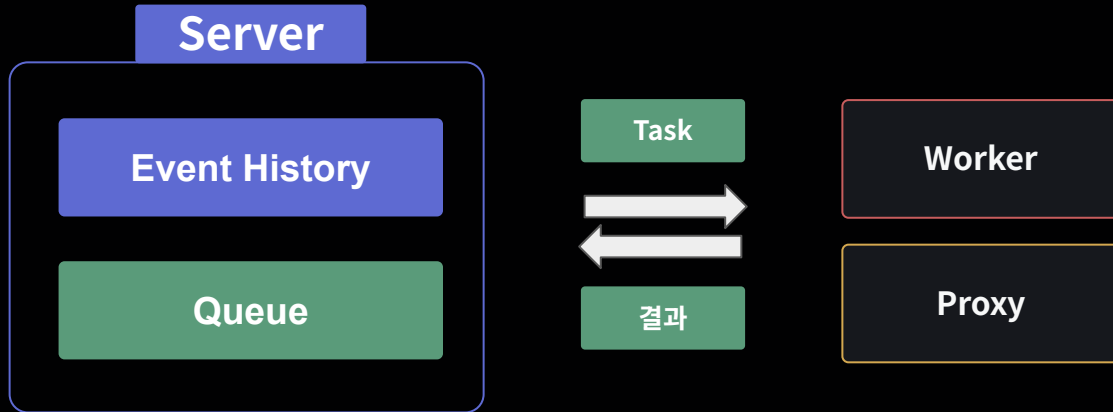
MyWorkflow로 보는 Durable Execution



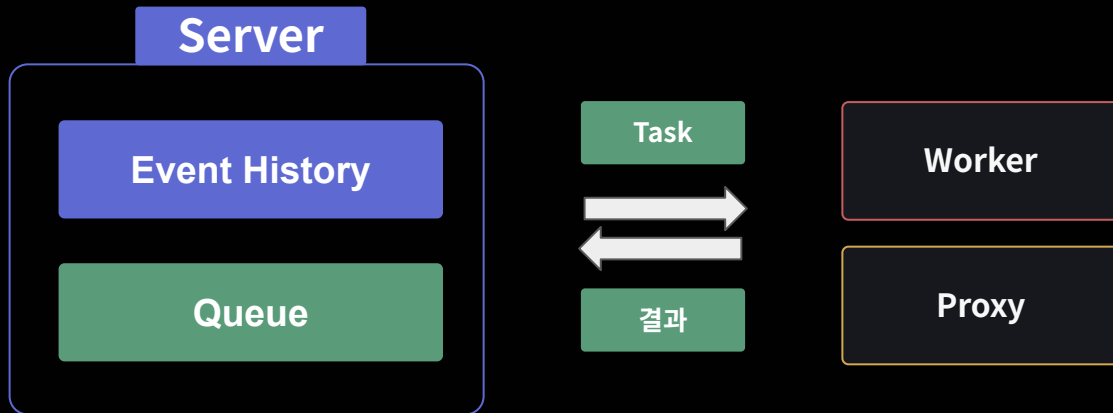
MyWorkflow로 보는 Durable Execution



MyWorkflow로 보는 Durable Execution

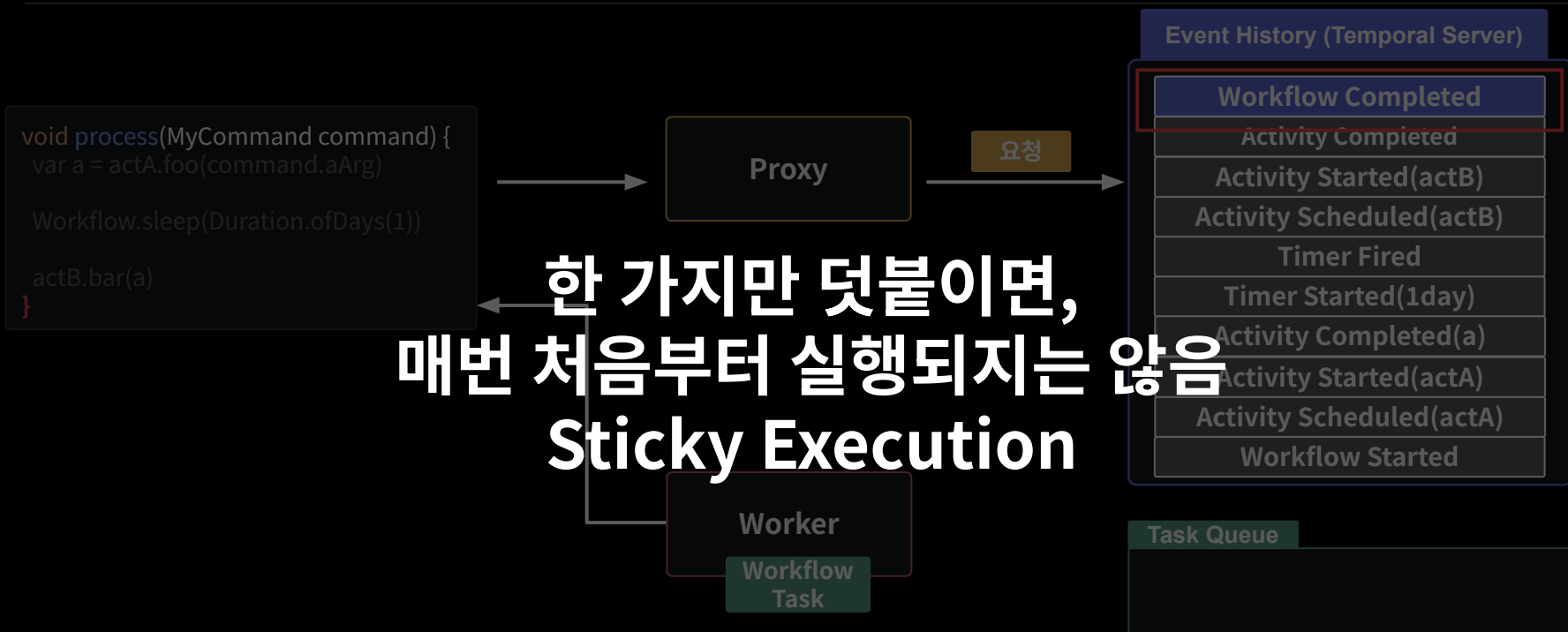


MyWorkflow로 보는 Durable Execution



어떤 상황이든 같은 매커니즘으로 동작
성공이든 실패든 끝을 맺는다

MyWorkflow로 보는 Durable Execution



목차

01

워크플로우는 왜 항상 복잡할까

고객 보상 시나리오 - 비즈니스 코드가 파묻히는 과정

02

Temporal이 제안하는 방식

실패가 없는 것처럼 짜고 나머지는 Temporal이 맡는다

03

어떻게 가능한가? (Durable Execution의 원리)

Event History · Replay · Proxy — 멈췄다 그 자리에서 이어지는 원리

04

그래서 무엇이 남았나

비즈니스 시맨틱만 남은 코드와 복잡도의 행방

그래서 무엇이 남았나?

비즈니스를 완수하기 위한 코드, 다 사라졌습니다

All or Nothing

타이머

타임아웃

재시도

보상

신호 처리

결과의 영속화

그래서 무엇이 남았나?

대신, 비즈니스만 남았습니다

- 비즈니스만 남은 코드
- 역할 · 책임 분리는 늘 해오던 것
- 기존에 알던 분리와는 다른 결

```
void process() {  
    // 데이터 수집  
    var data = collectData()  
    // 금액 평가  
    var credit = evaluateCredit(data)  
    // 7일 대기  
    Workflow.await(Duration.ofDays(7), () -> approve)  
  
    try {  
        // 장부 기록 및 지급  
        executeTransaction()  
    } catch (Exception e) {  
        // Transaction 롤백  
        rollback()  
    }  
}
```


그래서 무엇이 남았나? - 기존 워크플로우

분리를 아무리 잘 해도 워크플로우는 복잡하다

- 함수를 쪼개고
 - 진행 상태를 저장하고
 - 이미 진행된 상태인가 조건 분기
 - 재시도, 제한 시간 계산이 어렵
 - 비즈니스 보다 부가적인 코드가 더 드러남
-

그래서 무엇이 남았나?

한 함수에 비즈니스만 남기는 건 거의 불가능하다

- 아무리 잘 나뉘도 한 함수에 비즈니스 시멘틱만 남기는 것은 매우 어려움
- Temporal은 가능

```
void process() {  
    // 데이터 수집  
    var data = collectData()  
    // 금액 평가  
    var credit = evaluateCredit(data)  
    // 7일 대기  
    Workflow.await(Duration.ofDays(7), () -> approve)  
  
    try {  
        // 장부 기록 및 지급  
        executeTransaction()  
    } catch (Exception e) {  
        // Transaction 롤백  
        rollback()  
    }  
}
```

해결책 - 첫번째, 함수의 종단을 다루는 방법

Temporal: Event History + Replay (Event Sourcing)

```
void doStep1() {  
    // do step1  
}
```

```
void doStep2() {  
    // do step2  
}
```

**Workflow 조정자가
분리된 각 Step을 직접 호출**

```
void process(Context context) {  
    var status = getStatus(context.id)  
  
    if (status == Status.STEP1) {  
        doStep1()  
        status = updateStatus(Status.STEP2, context.id)  
    }  
  
    if (status == Status.STEP2) {  
        doStep2()  
        status = updateStatus(Status.DONE, context.id)  
    }  
}
```

**코드 내에서 저장된
상태에 따라 Step 선택**

해결책 - 첫번째, 함수의 중단

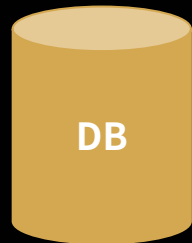
Durable Execution - 실행 그 자체가 상태

해결책 - 첫번째, 함수의 중단

Durable Execution - 실행 그 자체가 상태

기타 Workflow Engine

```
void process(Context context) {  
    var status = getStatus(context.id)  
  
    if (status == Status.STEP1) {  
        doStep1()  
        status = updateStatus(Status.STEP2,  
            context.id)  
    }  
  
    if (status == Status.STEP2) {  
        doStep2()  
        status = updateStatus(Status.DONE,  
            context.id)  
    }  
}
```



current_step
retry_count

해결책 - 첫번째, 함수의 중단

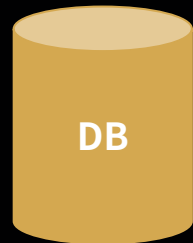
Durable Execution - 실행 그 자체가 상태

Temporal

```
void process() {  
  // 데이터 수집  
  var data = collectData()  
  // 금액 평가  
  var credit = evaluateCredit(data)  
  // 7일 대기  
  Workflow.await(Duration.ofDays(7),  
    () -> approve)  
  
  try {  
    // 장부 기록 및 지급  
    executeTransaction()  
  } catch (Exception e) {  
    // Transaction 롤백  
    rollback()  
  }  
}
```

기타 Workflow Engine

```
void process(Context context) {  
  var status = getStatus(context.id)  
  
  if (status == Status.STEP1) {  
    doStep1()  
    status = updateStatus(Status.STEP2,  
      context.id)  
  }  
  
  if (status == Status.STEP2) {  
    doStep2()  
    status = updateStatus(Status.DONE,  
      context.id)  
  }  
}
```

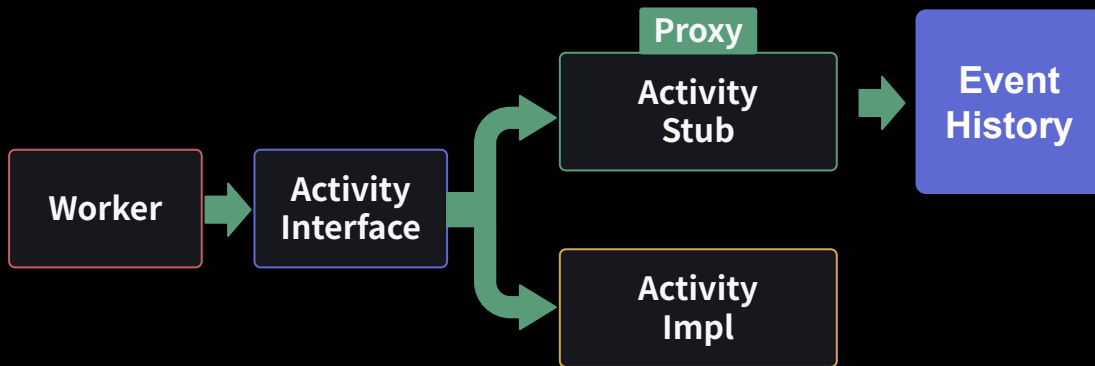


current_step
retry_count

해결책 - 두번째, Proxy

런타임에 끼워넣기

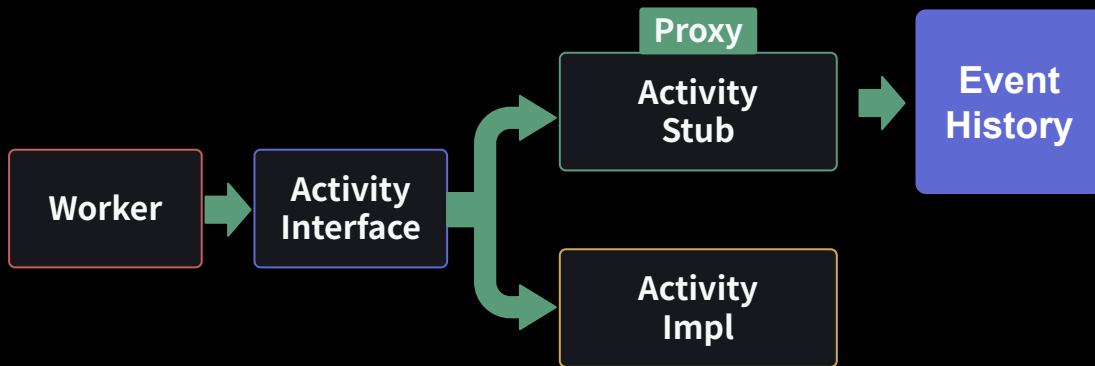
```
try {  
    ...  
    // 고객 지급  
    var receipt = transAct.payCredit(customerId,  
    amount)  
    ...  
} catch (Exception e) {  
    ...  
}
```



해결책 - 두번째, Proxy

런타임에 끼워넣기

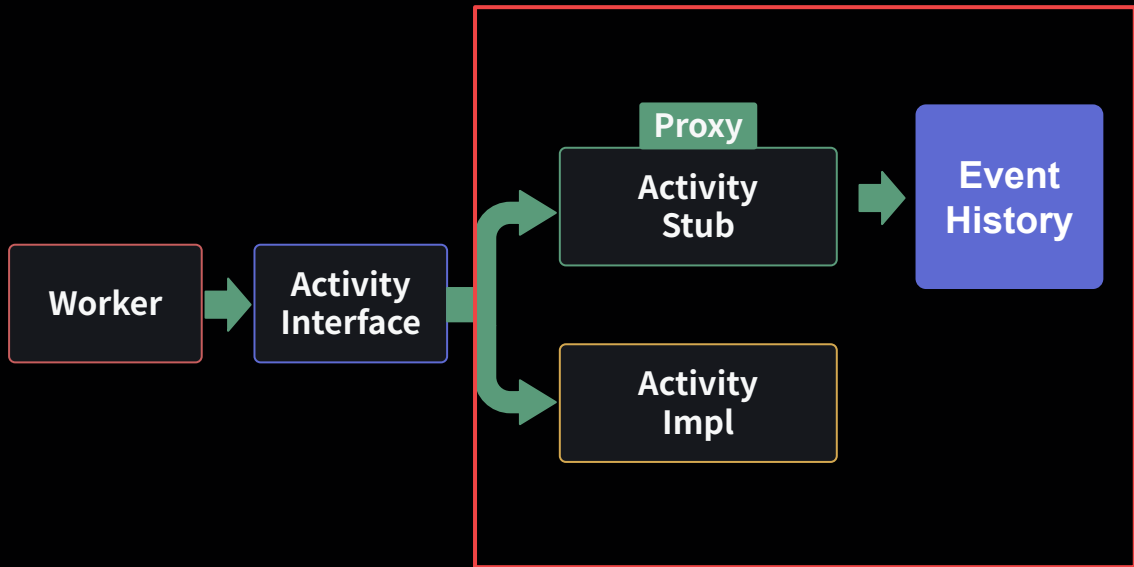
```
try {  
    ...  
    // 고객 지급  
    var receipt = transAct.payCredit(customerId,  
    amount)  
    ...  
} catch (Exception e) {  
    ...  
}
```



해결책 - 두번째, Proxy

런타임에 끼워넣기

```
try {  
  ...  
  // 고객 지급  
  var receipt = transAct.payCredit(customerId,  
    amount)  
  ...  
} catch (Exception e) {  
  ...  
}
```



그래서 무엇이 남았나? - 정리

| 경계가 코드에서 런타임으로

그래서 무엇이 남았나? - 정리

| 경계가 코드에서 런타임으로

실행 경계

- Event History + Replay

그래서 무엇이 남았나? - 정리

경계가 코드에서 런타임으로

실행 경계

- Event History + Replay

관심사 경계

- Proxy
-

그래서 무엇이 남았나? - 결과

| 우리 코드에는 비즈니스 시맨틱만 남는다

그래서 무엇이 남았나? - 결과

우리 코드에는 비즈니스 시맨틱만 남는다

본질적 복잡도

- 산정, 승인, 지급 - 문제 그 자체

그래서 무엇이 남았나? - 결과

우리 코드에는 비즈니스 시맨틱만 남는다

본질적 복잡도

- 산정, 승인, 지급 - 문제 그 자체

우발적 복잡도

- 재시도, 체크포인트, 재개분기 - 분산 환경이 불안정해서 생긴 것
-

그래서 무엇이 남았나? - 결과

우리 코드에는 비즈니스 시맨틱만 남는다

본질적 복잡도

- 산정, 승인, 지급 - 문제 그 자체

우발적 복잡도

- 재시도, 체크포인트, 재개분기 - 분산 환경이 불안정해서 생긴 것

소스 코드에 존재하지 않게 만듦 => 코드가 비즈니스 프로세스와 1:1

그래서 무엇이 남았나? - 결과

우리가 안 짤 뿐, 복잡한 코드는 Temporal 안에

- Temporal의 런타임으로 이동
- Proxy, Event Sourcing 처럼 이미 아는 것들의 조합으로 달성
- 그리고 Temporal에서 본질적 복잡도가 됨

각자 여러번 푼 문제를 한 곳에서 제대로 푼 것이 이동의 진짜 의미

그래서 무엇이 남았나? - 결과

물론 공짜는 아니다

- Temporal Server Cluster 운영 or Cloud 비용
 - 결정성, Replay 멘탈모델 학습
 - 워크플로우 코드는 결정성이라는 제약
 - replay를 위한 워크플로우 버저닝
 - 등등
-

그래서 무엇이 남았나? - 결과

워크플로우 코드 왜 항상 복잡할까?

- 분산환경이 만들어낸 우발적 복잡도를 직접 해소해와서
 - 비즈니스와 복잡도는 떼어내질 수 있다
 - 마법이 아니라 우리가 이미 아는 것들의 조합
 - 가져다 쓰든, 직접 만들든 우리의 선택
-

감사합니다.